



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



APLICACIÓ DE GESTIÓ PER A ENTRENADORS DE FUTBOL

MARC TURU ROCA

Thesis supervisor

XAVIER ORIOL HILARI (Department of Service and Information System Engineering)

Degree

Bachelor's Degree in Informatics Engineering (Software Engineering)

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

20/01/2025

Abstract

Technology is increasingly present in many aspects of our daily lives. From the everyday use of our mobile devices to advanced applications like artificial intelligence algorithms predicting diseases. To a greater or lesser extent, it helps us move forward and offers groundbreaking solutions, even in traditionally static domains.

One such area is football teams, where outdated and ineffective practices have persisted over the years. Despite being a globally influential sport, it has struggled to fully embrace the potential that it should have had in terms of modernization.

That is why **Let's Manage!** introduces an innovative and unique approach.

It aims to redefine the organization of football teams and what it implies in a new and efficient way, enhancing player performance while addressing their individual needs. This way, their qualities would be improved and they would facilitate indirect feedback for coaches and their teams.

Resum

La tecnologia és cada vegada més present en molts aspectes de la nostra vida quotidiana. Des de l'ús diari dels nostres telèfons mòbils fins a aplicacions avançades com els algorismes d'intel·ligència artificial que prediuen malalties. En major o menor mesura, ens ajuda a avançar i ofereix solucions innovadores, fins i tot en camps tradicionals.

Una d'aquestes àrees són els equips de futbol, on les pràctiques obsoletes i ineficaces han persistit al llarg dels anys. Tot i ser un esport amb influència mundial, ha lluitat per abraçar el potencial que hauria d'haver tingut en termes de modernització.

És per això que **Let's Manage!** introdueix un enfocament innovador i únic.

Pretén redefinir la organització dels equips de futbol i el que implica d'una manera eficaç, millorant el rendiment dels jugadors i adreçant les seves necessitats individuals. D'aquesta manera, es millorarien les seves qualitats i facilitarien el *feedback* indirecte pels entrenadors i els seus equips.

Resumen

La tecnología está cada vez más presente en muchos aspectos de nuestra vida cotidiana. Desde el uso diario de nuestros teléfonos móviles hasta aplicaciones avanzadas como los algoritmos de inteligencia artificial que predicen malaltias. En mayor o menor medida, nos ayuda a avanzar y ofrece soluciones innovadoras, hasta en campos tradicionales.

Una de estas áreas son los equipos de fútbol, donde las prácticas obsoletas e ineficaces han persistido a lo largo de los años. A pesar de que es un deporte con influencia mundial, ha luchado para abrazar el potencial que debería haber tenido en términos de modernización.

Es por esta razón que **Let's Manage!** introduce un enfoque innovador y único.

Pretende redefinir la organización de los equipos de fútbol y lo que implica de una manera eficaz, mejorando el rendimiento de los jugadores y dirigiendo sus necesidades individuales. De esta manera, se mejorarían sus cualidades y facilitarían el *feedback* indirecto para los entrenadores y sus equipos.

Contents Index

1. Introduction and contextualization.....	8
1.1 Context.....	8
1.2 Problem to be solved.....	8
1.3 Justification.....	9
1.4 Scope.....	11
1.4.1 Objectives and sub-objectives.....	11
1.4.2 Requirements.....	11
1.4.2.1 Functional requirements.....	11
1.4.2.2 Non-Functional requirements.....	12
1.4.3 Potential obstacles and risks.....	13
1.4.4 Alternative plans and obstacles for risks.....	14
1.5 Definitions.....	15
2. Specification.....	16
2.1 Stakeholders.....	16
2.2 Use cases.....	16
2.2.1 Use cases description.....	18
2.3 Conceptual data model.....	30
3. Design.....	32
3.1 Architecture description.....	32
3.1.1 Infrastructure architecture.....	32
3.1.2 Technologies and Services.....	32
3.2 Detailed design.....	33
3.2.1 Architectural pattern.....	33
3.2.2 Class design model.....	34
4. Implementation.....	36
4.1 Backend.....	36
4.1.1 Backend structure.....	36
4.1.2 Database interaction.....	36
4.1.3 API Endpoints.....	40
4.2 Frontend.....	44
4.2.1 Frontend structure.....	44
4.2.2 API Communication.....	44
4.3 User interface.....	49
4.3.1 Main Page.....	49
4.3.2 Register.....	50
4.3.3 Log in.....	50

4.3.4 Home.....	51
4.3.5 Profile.....	51
4.3.6 Clubs.....	52
4.3.7 Exercise(s).....	52
4.3.8 Team(s).....	53
4.3.9 Player(s).....	54
4.3.10 Match(es).....	55
4.3.11 Training(s).....	56
5. Testing.....	57
5.1 API endpoint tests.....	57
5.1.1 GetAll (Club).....	57
5.1.2 GetById (User).....	57
5.1.3 Create (User).....	57
5.1.4 Update (User).....	58
5.1.5 Delete (User).....	58
5.1.6 Verify user credentials.....	58
5.1.7 GetAllByType (Exercise).....	58
5.1.8 GetAllByPersonalized (Exercise).....	59
5.1.9 GetAllByTypeAndPersonalized (Exercise).....	59
5.1.10 GetAllByTeam (Player).....	59
5.1.11 GetAllByUser (Match).....	60
5.1.12 GetAllByUserAndTeam (Match).....	60
5.2 Test scenarios.....	61
5.2.1 Positive scenarios.....	61
5.2.2 Negative scenarios.....	62
6. Project management.....	65
6.1 Methodology and rigor.....	65
6.2 Initial tasks management.....	67
6.2.1 Project management.....	67
6.2.2 Project development.....	68
6.2.3 Resources.....	69
6.2.4 Tasks summary.....	70
6.2.5 Gantt chart.....	72
6.3 Initial budget management.....	72
6.3.1 Staff costs by activity.....	72
6.3.2 Generally calculated costs.....	74
6.3.3 Contingencies.....	76
6.3.4 Incidentals.....	76
6.3.5 Total cost estimation.....	77

6.4 Management control.....	77
6.5 Sustainability report.....	78
6.5.1 Self-assessment.....	78
6.5.2 Economic dimension.....	78
6.5.3 Environmental dimension.....	79
6.5.4 Social dimension.....	79
7. Deviations from the initial plan.....	81
7.1 Task deviations.....	81
7.2 Temporal deviations.....	81
7.3 Economic deviations.....	82
7.4 Final assessment.....	83
8. Laws and regulations.....	84
9. Knowledge integration.....	85
9.1 Knowledge acquired from the courses.....	85
9.2 Software Engineering specialization justification.....	86
9.3 Technical competences.....	86
10. Conclusions.....	88
10.1 Objectives conclusions.....	88
10.2 Future project.....	89
10.3 Final conclusions.....	89
11. References.....	91

Figures Index

Figure 1. Use cases diagram. The author.....	17
Figure 2. Conceptual data model. The author.....	30
Figure 3. Physical architecture model. The author.....	32
Figure 4. Class design model. The author.....	34
Figure 5. Main page. The author.....	49
Figure 6. Register page. Source: The author.....	50
Figure 7. Login page. Source: The author.....	50
Figure 8. Home page. Source: The author.....	51
Figure 9. Profile page. Source: The author.....	51
Figure 10. Clubs page. Source: The author.....	52
Figure 11. Exercises page. Source: The author.....	52
Figure 12. Exercise page. Source: The author.....	52
Figure 13. Teams page. Source: The author.....	53
Figure 14. Team page. Source: The author.....	53
Figure 15. Rival teams page. Source: The author.....	53
Figure 16. Rival team page. Source: The author.....	54
Figure 17. Players page. Source: The author.....	54
Figure 18. Player page. Source: The author.....	55
Figure 19. Matches page. Source: The author.....	55
Figure 20. Match page. Source: The author.....	55
Figure 21. Training page. Source: The author.....	56
Figure 22. Training page. Source: The author.....	56
Figure 23. 200 OK Response. Source: The author.....	61
Figure 24. 201 Created Response. Source: The author.....	61
Figure 25. 400 Bad request Response. Source: The author.....	62
Figure 26. 401 Unauthorized Response. Source: The author.....	62
Figure 27. 404 Not Found Response. Source: The author.....	63
Figure 28. 409 Conflict Response. Source: The author.....	63
Figure 29. 422 Unprocessable entity Response. Source: The author.....	64
Figure 30. 500 Internal Server Error Response. Source: The author.....	64

Figure 31. Scrum basic process. Source: [7].....	65
Figure 32. Taiga project management example. Source: [9].....	66
Figure 33. Github branches distribution. Source: [10].....	66
Figure 34. Gantt chart. Source: The Author.....	72

Tables Index

Table 1. Competence applications comparison. Source: The Author.....	10
Table 2. Risks, their probability and hours that would take. Source: The Author.....	14
Table 3. Human resources involved in the project with ID, Resource & Person. Source: The Author.....	69
Table 4. Material resources used in the project with ID, Resource & Why. Source: The Author.....	70
Table 5. Task summary with Code, Task, Workload, Dependencies, Resources and Sprint. Source: The Author.....	71
Table 6. Roles with gross salaries plus social security per hour. Source: The Author.....	73
Table 7. Personnel costs per activity (CPA). Source: The Author.....	74
Table 8. Hardware components costs. Source: The Author.....	75
Table 9. Software components costs. Source: The Author.....	75
Table 10. Generic cost (CG). Source: The Author.....	76
Table 11. Contingency cost. Source: The Author.....	76
Table 12. Incidentals cost. Source: The Author.....	77
Table 13. Total final cost. Source: The Author.....	77

1. Introduction and contextualization

The project *Management Application for Football Coaches* is done in the bachelor's thesis of the Computer Engineering Degree, more specifically in the Software Engineering branch in the Facultat d'Informàtica de Barcelona (FIB) from the Universitat Politècnica de Catalunya (UPC).

It belongs to the modality A and is directed by Xavier Oriol Hilari, PhD Software Engineering Researcher and part of the Department of Information Services and Systems Engineering at FIB.

Its primary goal is to put into practice the vast majority of aspects learned throughout the degree and develop a fully functional and practical web app.

1.1 Context

The difficult management of football teams has led many coaches to try to find ways to face complicated situations within the collective and individual ambits of these teams.

From awkward iPhone notes to keep important information about the players, matches and even training to impractical Excels sheets to keep track of players' minutes played. Although these are not bad options they are not as useful as they could be if we focus on the basic aspects that surround football and, more importantly, being a coach.

In this thesis, we are going to develop a functional and practical web application that can solve the problems raised previously and even more.

This project not only wants to develop a web application to make the life of coaches easier but also aims to be a new model of understanding a football team in terms of cohesion, training and improvement. It intends to serve as a model to enhance the improvement of football teams using statistics and learning from previous studies and practices.

1.2 Problem to be solved

Apart from common problems that coaches tend to find, as seen in this article [1], the football coaching world is outdated and antiquated, as it does not offer functional solutions to problems that, unlike it, remain more relevant than ever.

From tedious ways to address personal physical needs to obsolete training methods and match approaches. Coaches do not have a practical tool that can help them in management aspects that they can not control or tend to not do so. These issues prevent coaches, players, and teams from evolving within the world of sports and enhancing their virtues, which would lead them to enjoy much more the sport they practice and form part of.

Thus, putting into practice the concepts explained before and other useful as well as innovative ways of work, coaches will find remedies for the challenges that they have to face on a daily basis.

1.3 Justification

Before delving into the project, we must take into consideration important aspects such as market analysis and different studies that have been carried out. These topics will help us focus on the problem we intend to solve and how we want to do it.

That is why to carry out the market study, a table with the most relevant characteristics has been built. Then, comparing the several options presented to the one proposed in the document, we will be able to see how useful the app could be, what will be included and what features would be better to exploit in order to get the maximum and most useful performance from the application.

	LetsManage!	PizarraTáctica (Demo)	FutApp (Demo)	Gol manager	BCoach
Training planning	✓	✓	✓	✓	✓
Lineups creation	✓	✓	✓	✓	✓
Updates after match	✓	✗	✓	✓	✓
Player/team statistics	✓	✗	✓	✓	✓
Complex and detailed register	✗	✗	✓	✓	✓
Limited by free use	✗	✓	✓	✗	✗
Player evolution	✓	✗	✗	✗	✗
Personalized trainings by performance and necessities	✓	✗	✗	✗	✗
Rewards system	✓	✗	✗	✗	✗

Physical and health advice					
Advertisements					

Table 1. Competence applications comparison. Source: The Author.

Firstly, and having studied the current market situation, it has been decided to develop an application that includes the majority of basic aspects from a simple one: training, matches, lineups and player/team statistics. In nearly all the applications visited, these basic characteristics that any coach should take into account (whether or not they use directed applications) are included.

However, having seen the opportunities for improvement, **LetsManage!** intends to be not only a help tool for coaches but a new way to understand the player's needs and reward their effort. In this way, and empowering an infinite loop of feedback, the coach is nourished by the player's effort and the player does so thanks to the recognition and appreciation received. This strengthens the coach-player bond and the general cohesion of the team.

Moreover, the application plans to offer some extra content for players in an effort to keep them healthy. Therefore, even though the players are not in the official training, they can even stay in shape and ready for whatever comes in the future and can help them to maintain a healthy lifestyle. Although these features do not seem that important and related with coaches at first glance, they will maximize the players performance and will be an extra that even the players' families will appreciate.

Finally, taking a look at the table and taking into consideration the explanations, the viability of the project and the reason for the decision made are demonstrated because it not only offers good help for coaches but also additional characteristics that would be very helpful for any team and attractive for coaches of all kinds of levels and ages.

1.4 Scope

The application, rather than being ambitious because it wants to cover a lot, is ambitious because of what it wants to cover.

The User Interfaces are very common and standard which is a noteworthy point, as the application is addressed to all types of football coaches and there is a vast majority of them that are not that young so the final clients will appreciate a user-friendly application as well as one that solves many present problems.

1.4.1 Objectives and sub-objectives

The main objective of this project is to develop a fully functional web application used as a helpful tool for football coaches in scopes like training, matches, team information and statistics that facilitates the effective management of teams.

To achieve this objective, it can be broken down as follows:

Centralized data: Create a program that can offer access to data about teams, players, training, statistics and other information related to teams.

Performance analysis: Study the performance of the team and players individually in order to offer improvement decisions related to new and/or personalized training and match strategies.

Insights and conclusions generation: Create insights using statistical information in order to keep track of the work done and its evolution.

Training and match management: Create and update training to their liking and matches to simulate outcomings and possible situations.

1.4.2 Requirements

1.4.2.1 Functional requirements

The first part of requirements are the functional ones, those that refer to the usage of the application and the functionalities that it offers.

- **Match administration:** The user must be able to add, update and delete information from matches along the season.
- **Nutrition summaries:** The application must be able to present healthy plans that can be related to the players analyzing their requirements.
- **Player administration:** The user must be able to add, update and delete players from the teams that it is part of, as well as their information.

- **Reports:** The application must provide reports from time to time with useful team and performance information for the coach.
- **Statistics administration:** The user must be able to add information from players, matches, training and other useful aspects in order to obtain statistics from them in exchange.
- **Team administration:** The user must be able to add, update and delete football teams from which it is coach from and edit their information.
- **Training administration:** The user must be able to build and plan training sessions with specific drills and exercises.
- **User administration:** The user must be able to update and manage their profile to their liking.
- **User authentication:** The user must be able to log in and log out with a specific and unique username and password.
- **Search and filter tools:** The user must be able to search and filter to find information based on standards.

It is worth emphasizing that there may be functionalities that might change in the future, depending on the evolution of the development of the application.

1.4.2.2 Non-Functional requirements

The second part of requirements are the non-functional ones, those that refer to the quality of the application rather than specific tasks from it [3].

- **Capacity:** The application must certify the correct usage of large amounts of data like 50 active users concurrently without any inconvenience.
- **Documentation:** The application must be noted so developers have well organized information that can be helpful.
- **Learnability:** The application must be easy to understand and use so the end user can make useful and efficient use of it.
- **Performance:** The application must be able to respond to user interaction efficiently and with high performance to provide a fluid user experience.
- **Privacy:** The application must guarantee the privacy from the people to whom their information and data are stored as well as ensure that all the functionalities are available for users (and developers,... obviously).
- **Scalability:** The application must be prepared to handle an increasing amount of information from users and data in general.

- **Security:** The application must ensure the protection of data by applying protective controls and measures regarding confidentiality and integrity.
- **Usability:** The application must be user-friendly in terms of appearance and for the people that are supposed to use it.

1.4.3 Potential obstacles and risks

Although the project is expected to go as planned, there are always things that cause problems and delay the development of the application. That is why in this section potential obstacles and risks will be addressed and, as importantly, a margin of error estimation will be done so they don't catch us off guard.

Bugs: When developing an application, it is common to find bugs along the coding which can result in wastes of time. These bugs must be treated as soon as possible so they don't cause any other inconvenience.

Therefore, and by previous experience on projects worked on and taking into account the scope and duration of the project, I expect to have a margin of error of 15%, so I predict the 10%-15% of functionalities to have bugs that need treatment.

Delivery deadlines: As a final project (and even as an application development) there will be delivery deadlines that could cause problems when implementing all the functionalities for the application. For this reason I will give myself some extra weeks of margin of error so I will not be surprised by these deadlines.

Inexperience with technologies: In this application I will have to use a lot of technologies that will bring the application to life. From backend to frontend, from database to deployments, I'm not familiar with some of them. This is why, in the timeline I bear in mind the margin of time that will take me to get used to them.

Technical problems: Working on an application implies using or benefiting from PCs, electricity, Internet connection, monitors, peripherals and a workspace. That is why if some of these devices fail, it could set me back. However, I rarely expect this to happen as, where I live, these situations are not common. Nevertheless, I will always have a second option for every of the aspects mentioned above in case of failure so I can continue working.

1.4.4 Alternative plans and obstacles for risks

In order to present a good project, it is essential to take into account the risks and difficulties. That is why, in Table 2, a brief summary of the obstacles, their probability to occur and the approximate hours it would take to resolve them is presented.

The hours that would have to be used in case these risks occur would be borrowed from extra management tasks or added to the daily workload.

Risk	Probability	Hours
Bugs	High	25
Delivery deadlines	Medium	15
Inexperience with technologies	Low	10
Technical problems	Low	5

Table 2. Risks, their probability and hours that would take. Source: The Author.

Bugs are highly probable to occur so, to prevent them, tests will be done in order to examine each functionality. Moreover, these tests will allow me to isolate errors and treat them separately from other tasks, so no extra bugs appear.

If there are bugs that appear unexpectedly, tasks will be created to treat and solve them.

To avoid being short on **delivery** time, scrum planning will be followed and extra time will be left to finish the tasks that are not finished in each sprint, adding them to the next one and leaving a final one with extra time to finish them all. This way, if extra time is needed, it would not be a serious problem.

The vast majority of **technologies** used are familiar to me, since I have worked with them on other projects, which is why I do not expect them to be a great waste of time.

Despite this, if it were necessary for this possible obstacle, a brief study of these technologies would be carried out, not resulting in a big waste of time at all.

Technical problems do not commonly occur and the solution would be as easy as looking for another workspace to develop the app while they are solved, needing extra available resources from my family members like computers, peripherals and other technical items.

1.5 Definitions

In order to comprehend the project and its applications, there are some important concepts that are related with the topic and that the reader must be familiar with.

Some correspond to application conceptions, some others to team evaluations and some others to coaching methodologies:

Data-Driven Decision-Making: Approach that uses quantitative and qualitative data to guide important team and strategy decisions to depend less on unsafe perceptions and enhance good option choice.

Evidence-Based Coaching: Coaching methods that use empirical evidence to design personalized and useful training, ensuring that interventions are tailored to individual needs.

Football Benchmarking: Process used by teams to compare their practices and results with the ones from other ones in order to identify optimization areas and boost team performance/improvement.

Performance Analysis: It is the action of evaluating the efficiency and effectiveness of a process in order to extract valuable conclusions to improve it or do so with other elements related to it. By evaluating performance metrics and identifying patterns, long-term sustainability can be improved.

Periodization Methodology: Training strategy that divides the practices in cycles depending on the phase of the week/year in which the team finds itself. These distributions facilitate the *workload management* in an effort to optimize the performance in key moments of the season [4].

Tactical Analysis: Is the examination of specific actions and decisions made during strategies to achieve immediate objectives and how they can be adjusted allowing modifications based on the circumstances at hand.

User Interfaces (UI): User interface (to be more specific *Graphical User Interface*) is the set of visual elements that the user interacts with in order to make the application do the tasks that it is programmed to. These include windows, icons, buttons, menus and all these types of components.

Web Application: A web application is a type of app that can be accessed through a web browser and that is stored in a remote server.

2. Specification

2.1 Stakeholders

This section lists the different stakeholders of the project. Each diverse group brings unique perspectives that contribute to understand how people will take part on the development of the application:

Coaches: They will benefit from this application. These people will use it by preparing training, updating lineups and offering personalized practices for the players.

Competing companies: They will try to compete and take over the market by creating a better or similar application.

Cybersecurity and data encryption expert: It will help the company by protecting the system against security problems and preserving the user information.

Expert in data protection law: It will inform about state laws and will help the company to fulfill with the *Ley Orgánica de Protección de Datos Personales y Garantía De Los Derechos Digitales* [2] and the *General Data Protection Regulation*.

Football clubs: Thanks to the use of the application, player satisfaction will grow and parents will be more comfortable signing their children to clubs that use this tool.

Players: They will benefit indirectly from the application as their statistics will help the coach make better decisions and exploit their potential.

Treball de Fi de Grau Director: Xavier Oriol Hilari, PhD Software Engineering Researcher and responsible of guiding and coordinating the project.

Web development team: They will be the ones that will carry out the application code.

2.2 Use cases

To better understand the interaction between the user that uses the application and the system itself, use cases can be a good way to express different situations in which the user will have to execute functionalities from it.

In this way, use cases help us by presenting possible scenarios where actors¹ make use of functionalities given by triggers when preconditions are fulfilled. Although all the use cases are thought of being well-done, some extensions can be helpful to express possible errors and alternative ways of executing the functionality.

¹ The actor user refers to the coaches that will use the application

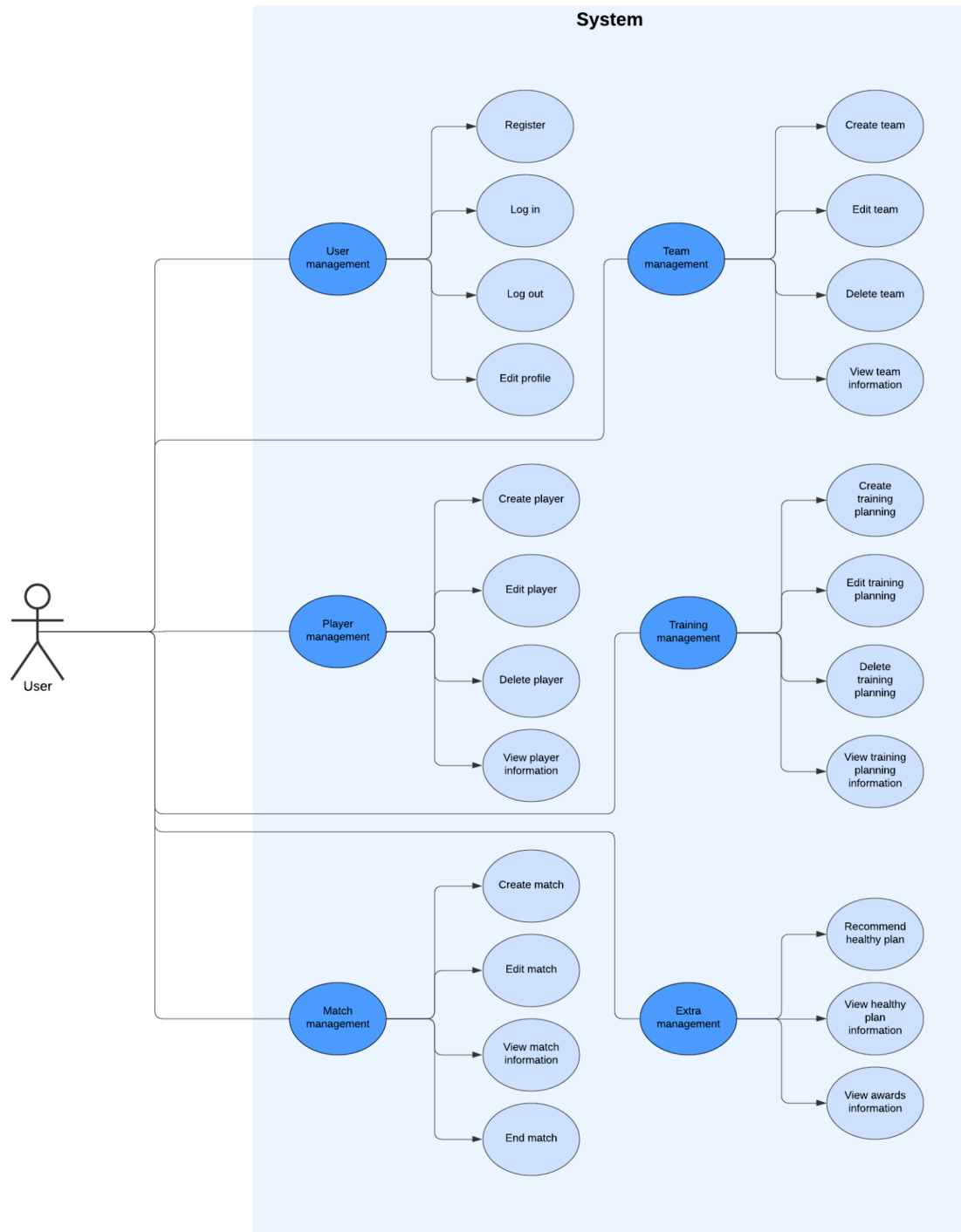


Figure 1. Use cases diagram

2.2.1 Use cases description

Use case	Register (UCR)
Actor	User.
Precondition	The user is not registered.
Trigger	The user wants to have an account.
Main success scenario	1. The user goes to the register page. 2. The user introduces a new username and password. 3. The user submits the form. 4. The system validates the form. 5. The system creates a new profile with the indicated parameters.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing. 4a. The system detects that there is already a user with that username. 4a1. The system cancels the registration and reports the error to the user. 4b. The system detects that there are not enough characters or there are offensive terms. 4b1. The system cancels the registration and reports the error to the user.

Use case	Log in (UCLI)
Actor	User.
Precondition	The user is registered.
Trigger	The user wants to use the application.
Main success scenario	1. The user goes to the login page. 2. The user introduces its username and password.

	3. The user submits the form. 4. The system validates the form. 5. The user accesses the application with its account.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing. 4a. The system detects that there is not a user with that username registered. 4a1. The system cancels the login and reports the error to the user. 4b. The system detects that the password is not the same as the user registered. 4b1. The system cancels the login and reports the error to the user.

Use case	Log out (UCLO)
Actor	User.
Precondition	The user is logged in.
Trigger	The user wants to end its session.
Main success scenario	1. The user goes to the profile page. 2. The user logs out with the option presented by the application.
Extensions	-

Use case	Edit profile (UCEPr)
Actor	User.
Precondition	The user is logged in.
Trigger	The user wants to update its profile.
Main success scenario	1. The user goes to the profile page. 2. The user chooses to edit its profile with new information.

	3. The user submits the form. 4. The system validates the form. 5. The system updates the profile information.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing. 4a. The system detects that there are not enough characters or there are offensive terms. 4a1. The system cancels the update and reports the error to the user.

Use case	Create team (UCCT)
Actor	User.
Precondition	The user is logged in.
Trigger	The user wants to create a new team.
Main success scenario	1. The user goes to the my teams page if he wants to create one of its teams or to the rival teams page if he wants to create one of its rival teams. 2. The user chooses to create a team with its information. 3. The user submits the form. 4. The system validates the form. 5. The system creates the team.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing. 4a. The system detects that the MyTeam/RivalTeam created already exists. 4a1. The system cancels the creation and reports the error to the user.

Use case	Edit team (UCET)
Actor	User.
Precondition	The user is logged in. The user has at least one team.
Trigger	The user wants to edit one of its teams.
Main success scenario	1. The user goes to the my teams page if he wants to edit one of its teams or to the rival teams page if he wants to edit one of its rival teams. 2. The user chooses to edit a team with new information. 3. The user submits the form. 4. The system validates the form. 5. The system updates the team information.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing.

Use case	Delete team (UCDT)
Actor	User.
Precondition	The user is logged in. The user has at least one team.
Trigger	The user wants to delete one of its teams.
Main success scenario	1. The user goes to the my teams page if he wants to delete one of its teams or to the rival teams page if he wants to delete one of its rival teams. 2. The user chooses to delete one of its teams. 3. The system deletes the team.
Extensions	-

Use case	View team information (UCVPI)
Actor	User.
Precondition	The user is logged in. The user has at least one team.
Trigger	The user wants to check team information.
Main success scenario	1. The user goes to the my teams page if he wants to review one of its teams or to the rival teams page if he wants to review one of its rival teams. 2. The user chooses to view a team. 3. The system reviews the team's information, characteristics and stats.
Extensions	-

Use case	Create player (UCCP)
Actor	User.
Precondition	The user is logged in. The user has at least one team.
Trigger	The user wants to create a new player.
Main success scenario	1. The user goes to the players page. 2. The user chooses to create a player with its information. 3. The user submits the form. 4. The system validates the form. 5. The system creates the player.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing. 4a. The system detects that there is already a player with that name. 4a1. The system cancels the registration and reports the error to the user.

Use case	Edit player (UCEPI)
Actor	User.
Precondition	The user is logged in. The user has at least one player.
Trigger	The user wants to edit one of its players.
Main success scenario	1. The user goes to the players page. 2. The user chooses to edit one of its players with new information. 3. The user submits the form. 4. The system validates the form. 5. The system updates the player information.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing.

Use case	Delete player (UCDP)
Actor	User.
Precondition	The user is logged in. The user has at least one player.
Trigger	The user wants to delete one of its players.
Main success scenario	1. The user goes to the players page. 2. The user chooses to delete one of its players. 3. The system deletes the player.
Extensions	-

Use case	View player information (UCVPI)
Actor	User.
Precondition	The user is logged in. The user has at least one player.
Trigger	The user wants to check the player's information.
Main success scenario	1. The user goes to the players page. 2. The user chooses to view one of its players. 3. The system reviews player's information, characteristics and stats.
Extensions	-

Use case	Create training planning (UCCTP)
Actor	User.
Precondition	The user is logged in. The user has at least one team.
Trigger	The user wants to create a training plan.
Main success scenario	1. The user goes to its teams page. 2. The user selects the existing team to which it wants to create the training plan. 3. The user chooses to create a training plan with its information. 4. The user submits the form. 5. The system validates the form. 6. The system creates the training plan.
Extensions	4a. The user submits an incomplete form. 4a1. The system indicates to the user which information is missing.

Use case	Edit training planning (UCETP)
Actor	User.
Precondition	The user is logged in. The user has at least one training plan.
Trigger	The user wants to edit one of its training plans.
Main success scenario	1. The user goes to the training page. 2. The user chooses to edit one of its training plans with new information. 3. The user submits the form. 4. The system validates the form. 5. The system edits the training plan.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing.

Use case	Delete training planning (UCDTP)
Actor	User.
Precondition	The user is logged in. The user has at least one training plan.
Trigger	The user wants to delete one of its training plans.
Main success scenario	1. The user goes to the training page. 2. The user chooses to delete one of its training plans. 3. The system deletes the training plan.
Extensions	-

Use case	View training planning information (UCVTPI)
Actor	User.
Precondition	The user is logged in. The user has at least one training plan.
Trigger	The user wants to check training planning information.
Main success scenario	1. The user goes to the training page. 2. The user chooses to view one of its training plans. 3. The system reviews the training plan and its information.
Extensions	-

Use case	Create match (and receive strategy suggestions) (UCCM)
Actor	User.
Precondition	The user is logged in. The user has at least one team. The team has enough players to play.
Trigger	The user wants to create a match.
Main success scenario	1. The user goes to the team page. 2. The user selects the existing team to which it wants to create the match. 3. The user chooses to create a match with its information. 4. The user submits the form. 5. The system validates the form. 6. The system creates the match. 7. The system analyzes the opponent's team data. 8. The system analyzes the home's team data. 9. The system generates suggestions regarding the context studied. 10. The system displays the information.

Extensions	4a. The user submits an incomplete form. 4a1. The system indicates to the user which information is missing. 5a. The system detects that the teams are the same. 5a1. The system cancels the match creation and reports the error to the user.
-------------------	--

Use case	Edit match (and strategies) (UCEM)
Actor	User.
Precondition	The user is logged in. The user has at least one match.
Trigger	The user wants to edit one of its matches.
Main success scenario	1. The user goes to the matches page. 2. The user chooses to edit one of its matches with information and data provided by previous strategies suggestions. 3. The user submits the form. 4. The system validates the form. 5. The system edits the team.
Extensions	3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing.

Use case	View match information (UCVMI)
Actor	User.
Precondition	The user is logged in. The user has at least one match.
Trigger	The user wants to check match information.
Main success scenario	1. The user goes to the matches page. 2. The user chooses to review one of its matches.

	3. The system reviews the match and its information.
Extensions	-

Use case	End match (UCEM)
Actor	User.
Precondition	The user is logged in. The user has at least one match.
Trigger	The user wants to close a finished match.
Main success scenario	1. The user goes to the matches page. 2. The user chooses to close one of its matches and add post-match stats and information. 3. The user submits the form. 4. The system validates the form. 5. The system elaborates comparisons between pre-match analysis and post-match performance. 6. The system displays the information.
Extensions	2a. The match has no pre-match information (has not been played) 2a1. The system cancels the match closure and reports the error to the user. 3a. The user submits an incomplete form. 3a1. The system indicates to the user which information is missing.

Use case	Recommend healthy plan (UCGA)
Actor	User.
Precondition	The user is logged in. The user has at least one player.
Trigger	The user wants to recommend one healthy plan to a player.

Main success scenario	1. The user goes to the team page. 2. The user chooses one of its players. 3. The user studies the player's necessities and the healthy plans. 4. The user relates the player with the most appropriate healthy plan. 5. The system relates the player and the healthy plan.
Extensions	-

Use case	View healthy plan information (UCVHPI)
Actor	User.
Precondition	The user is logged in. The user has at least one healthy plan.
Trigger	The user wants to check healthy plan information.
Main success scenario	1. The user goes to the healthy plans page. 2. The user chooses to view one healthy plan. 3. The system reviews the healthy plan's information.
Extensions	-

Use case	View awards information (UCVAI)
Actor	User.
Precondition	The user is logged in.
Trigger	The user wants to check awards information.
Main success scenario	1. The user goes to the awards page. 2. The system reviews the awards' information.
Extensions	-

2.3 Conceptual data model

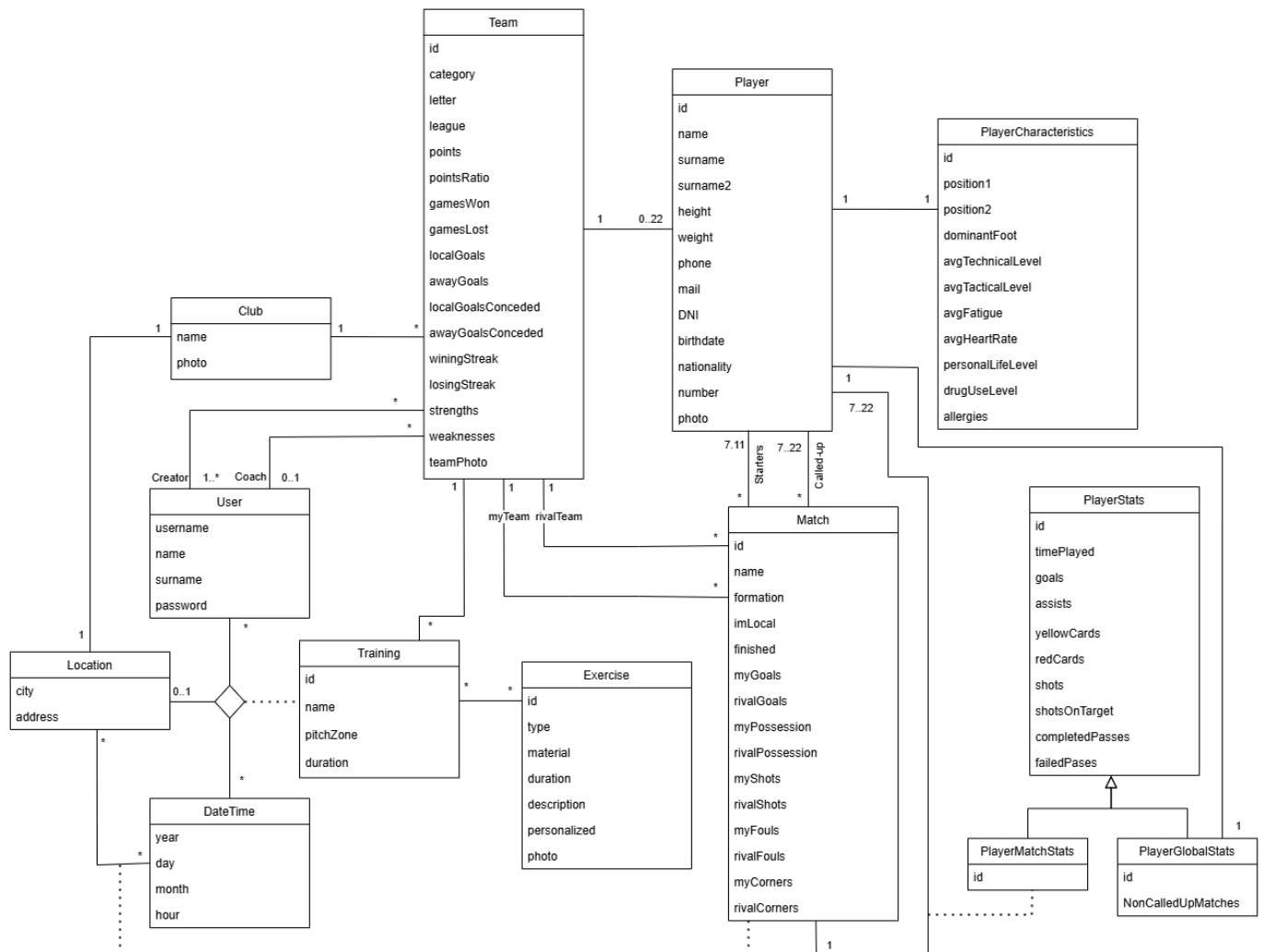


Figure 2. Conceptual data model

Textual Restrictions:

1. (**Club**, name), (**Team**, club+category+letter+creator), (**User**, username),
 (**Location**, city+address), (**DateTime**, year+month+day+hour),
 (**Training**, team+user+location+datetime), (**Exercise**, id),
 (**Player**, name+surname+surname2), (**PlayerCharacteristics**, player_”PlayerCharacteritics”),
 (**PlayerStats**, player_”PlayerStats”), (**Match**, myTeam+name)

2. If it is a football 7vs7 Team, the called-up players go between 7-16
3. If it is a football 11vs11 Team, the called-up players go between 11-22
4. If it is a football 7vs7 Team, the starters are 7.
5. If it is a football 11vs11 Team, the starters are 11.
6. The maximum yellow cards in PlayerMatchStats are 2.
7. The maximum red cards in PlayerMatchStats is 1.
8. The avgTechnicalLevel, avgTacticalLevel, avgFatigue, personalLifeLevel and drugUseLevel go from 0-10.
9. When creating a match, the myTeam and rivalTeam can not be the same team.
10. The photo from the Club and the teamPhoto from the Team are the same.
11. The duration from a Training is the sum of duration from its exercises.
12. The username, name and surname must be at least 2 characters long.
13. The password must be at least 8 characters long.
14. The PlayerStats attributes are the sum of all the MatchStats attributes from the player.
15. A Player can only be part of a Team that coaches the User.
16. The PlayerCharacteristics instance and the PlayerStats instance must belong to the same Player.
17. The Team from which the training is prepared must be the same as the Team that the user coaches.
18. 2 Players that belong to the same Team can not have the same number.
19. Each instance of the class Player must have a unique 'DNI', 'phone' and 'email'.
20. position1 and position2 from PlayerCharacteristics can not be the same.
21. shotsOnTarget <= shots.
22. The attribute pitchZone is unique.
23. There can not be 2 training sessions from the same team overlapping within 1:30 hours.
24. There can not be 2 matches from the same team overlapping within the time established depending on the team's category.

The PlayerCharacteristics (and PlayerStats) class has been separated from the Player's class so that it can be better appreciated and for its more practical use in the program, although they could have been integrated together.

I have made it possible for there to be 2 users in the same location with the same date so that, if a coach makes a mistake in the creation of one of their training sessions or matcehs, you don't not depend on their mistake.

3. Design

3.1 Architecture description

3.1.1 Infrastructure architecture

In order to develop a functional web application, structuring its infrastructure architecture is a must. This is why, before starting to write the code and other aspects, the following sequence that the technologies are going to follow has been thought:

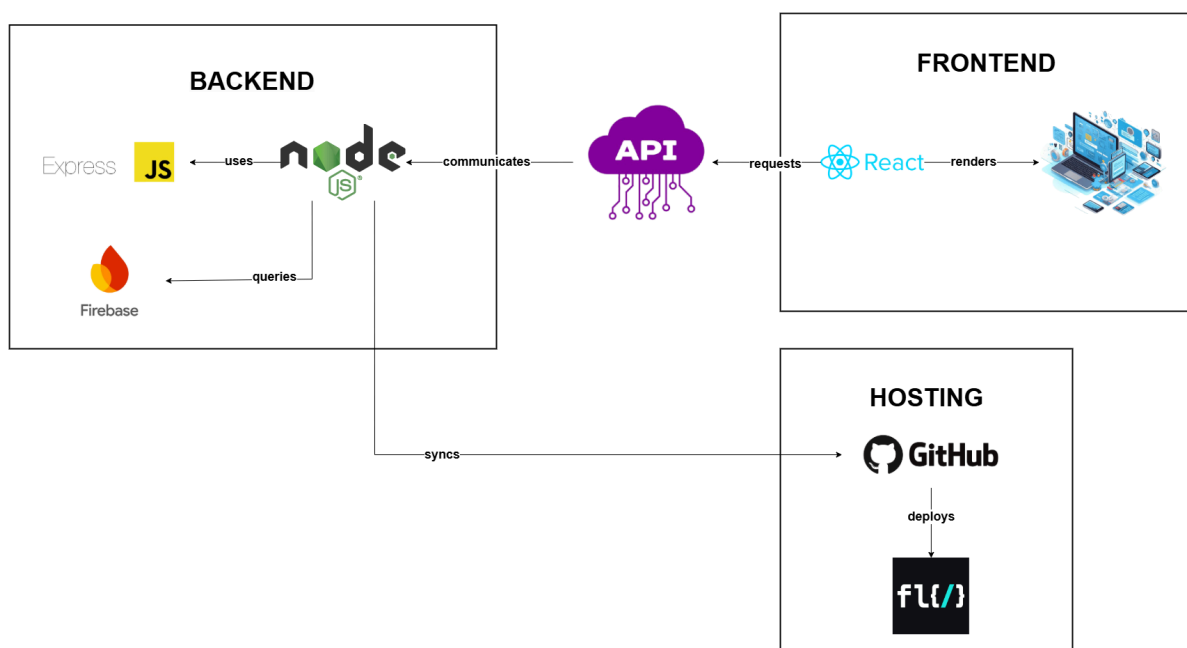


Figure 3. Physical architecture model

3.1.2 Technologies and Services

As I have done the project by myself, the technologies chosen have been the ones that I have already worked with because of the fact that I have liked the results after working with all of them.

By doing this, I prevent losing excessive time on learning new technologies and I can go straight to the point.

NodeJS: It is a programming environment based in JavaScript. With it, the backend server where the application will be executed will be established.

It has been chosen because of its simplicity and familiarity with.

ExpressJS: It is an open source framework for NodeJS used to develop web applications. With it, it will help NodeJS to communicate with the frontend with its API REST.

It has been chosen because of its familiarity with.

Firebase: It is a development platform from Google. With it, the database from where the data will be written and read will be built.

It has been chosen because of its familiarity with and because it is a NoSQL Database, which allows a more comfortable and flexible use.

React: The frontend app where the application will be presented has been decided to be available as a web application.

This is why, previously having used React Native, React seemed like a great option to develop the frontend part that later would be transformed in a web application format.

Github: With the usage of Github, the version control would be so much easier to do, as this hosting allows the user to create and delete new branches to separate the different functionalities to implement in an intuitive way. This is why, with its usage, the code is stored there and organised as planned.

F10: F10 has been chosen to be the deployment service because of its ease of use and because it is not mandatory to use the credit card.

As I am not very familiar with any deployment technology, the facilities that f10 provided were one of the points that I took into account the most. Moreover, I wanted the application to have a good response time for API calls and to be able to connect the Github repository where the code was stored.

3.2 Detailed design

3.2.1 Architectural pattern

The architectural pattern applied for designing the application has been the **Model-View-Controller**.

Models: They are composed of the classes with the information of the conceptual models and the logic from the functionalities to implement that will communicate with the database.

Controllers: They are responsible to connect the frontend calls with the backend implementations in the models. They go from frontend to backend to ask and from backend to frontend to answer.

Views: They are accountable to receive the information from the backend and render information for later viewing and user interaction.

3.2.2 Class design model

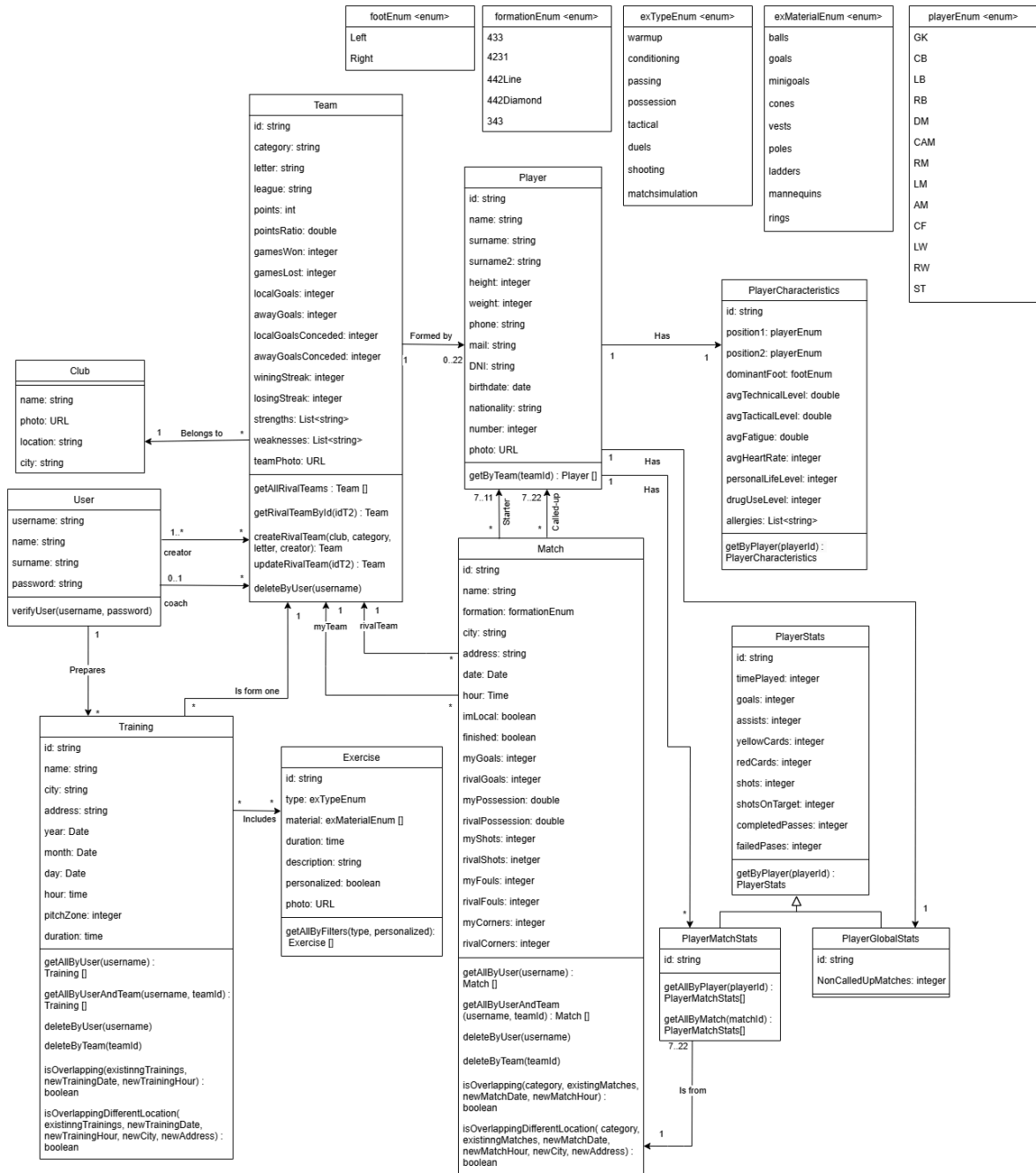


Figure 4. Class design model

Textual Restrictions:

1. (**Club**, name), (**Team**, id), (**User**, username), (**Training**, id), (**Exercise**, id), (**Player**, id), (**PlayerCharacteristics**, id), (**PlayerStats**, id), (**Match**, id)
2. If it is a football 7vs7 Team, the called-up players go between 7-16
3. If it is a football 11vs11 Team, the called-up players go between 11-22
4. If it is a football 7vs7 Team, the starters are 7.
5. If it is a football 11vs11 Team, the starters are 11.
6. The maximum yellow cards in PlayerMatchStats are 2.
7. The maximum red cards in PlayerMatchStats is 1.
8. The avgTechnicalLevel, avgTacticalLevel, avgFatigue, personalLifeLevel and drugUseLevel go from 0-10.
9. When creating a match, the myTeam and rivalTeam can not be the same team.
10. The photo from the Club and the teamPhoto from the Team are the same.
11. The duration from a Training is the sum of duration from its exercises.
12. The username, name and surname must be at least 2 characters long.
13. The password must be at least 8 characters long.
14. The PlayerStats attributes are the sum of all the MatchStats attributes from the player.
15. A Player can only be part of a Team that coaches the User.
16. The PlayerCharacteristics instance and the PlayerStats instance must belong to the same Player.
17. The Team from which the training is prepared must be the same as the Team that the user coaches.
18. 2 Players that belong to the same Team can not have the same number.
19. Each instance of the class Player must have a unique 'DNI', 'phone' and 'email'.
20. position1 and position2 from PlayerCharacteristics can not be the same.
21. shotsOnTarget <= shots.
22. The attribute pitchZone is unique.
23. There can not be 2 training sessions from the same team overlapping within 1:30 hours.
24. There can not be 2 matches from the same team overlapping within the time established depending on the team's category.

All classes are supposed to include the *create*, *update*, *delete*, *getAll*, *getById* and *exists* functions.

This design model reflects the changes made from the conceptual data model. N-ary associations and associative classes have been eliminated, making it a more detailed model in terms of attributes, functions, and the direction of associations..

4. Implementation

4.1 Backend

4.1.1 Backend structure

For the backend, the organization followed has been:

- **node_modules**: It contains all the installations and dependencies for the server and the libraries used, like *Express*, *Cors*, *firebase-admin*, *axios* and *nodemon* among others.
- **src**: It contains all the executable code that will be used to implement the application web.
 1. **Controllers**: All the classes that handle incoming requests from the frontend, send back the responses and handle possible errors.
 2. **Models**: All the classes that interact with the database and represent class models in code form.
 3. **Database**: All the creations, initializations, disconnections of the firebase database and the functions that are used to interact with it.
 4. **Routes**: The initialization of the Router used and the endpoint definitions with the HTTP methods used to communicate the backend and the frontend.
 5. **Schemas**: All the validation rules for the data used.
 6. **Tests**: All the tests for every HTTP method for every controller class in order to confirm the correct API endpoints behaviour.

4.1.2 Database interaction

To connect with the Firebase database and interact with it, apart from the installations, connections, desconnections, initializations required and the instances created, some basic functions have been added in order to exploit all the possible options that Firebase offers and that will help implement the functionalities from the app:

obtainDataCollection (getAll)

```
obtainDataCollection = async(collection) => {  
  try {  
    const collectionRef = db.collection(collection);  
    const snapshot = await collectionRef.get();  
    const data = [];  
  
    snapshot.forEach((doc) => {  
      data.push({  
        id: doc.id,
```

```

        ...doc.data(),
    });
});

return data;
} catch (error) {
    console.error('Error obtaining the data:', error);
    throw error;
}
};

```

- **collection**: Class to which the document belongs to.
- This function returns all the instances from a certain class.

In this function, an entire *collection* class instances from the database and their attributes are returned in JSON format.

obtainData (getById)

```

obtainData = async (collection, docID) => {
    try {
        const collectionRef = db.collection(collection);
        const docRef = await collectionRef.doc(docID).get();

        if (docRef.exists) {
            return {
                id: docRef.id,
                ...docRef.data(),
            };
        }
        else {
            throw new Error('Document does not exist');
        }
    } catch (error) {
        console.error('Error obtaining the data:', error);
        throw error;
    }
};

```

- **collection**: Class to which the document belongs to.
- **docID**: Primary key of the instance to search.

In this function, an instance identified by its *docID* from the class *collection* and its attributes are returned in JSON format.

obtainDataCollectionByField (getByField)

```
obtainDataCollectionByField = async (collection, fieldName, value) => {
  try {
    const collectionRef =
db.collection(collection).where(fieldName, "==", value);
    const snapshot = await collectionRef.get();
    const data = [];

    snapshot.forEach((doc) => {
      data.push({
        id: doc.id,
        ...doc.data(),
      });
    });
    return data;
  } catch (error) {
    console.error('Error obtaining the data:', error);
    throw error;
  }
};
```

- **collection**: Class to which the document belongs to.
- **fieldName**: Attribute of the instances to search.
- **value**: Property to compare in the attribute *fieldName* of all instances

In this function, the instances where the *fieldName* is equal to the *value* from the class *collection* and their attributes are returned in JSON format.

addNewData (create)

```
addNewData = async (collection, docID, newData) => {
  try {
    const collectionRef = db.collection(collection);
    const docRef = collectionRef.doc(docID);
    await docRef.set(newData);
    return docRef.id;
  } catch (error) {
    console.error('Error adding data', error);
  }
};
```

```

        throw error;
    }
};

```

- **Collection:** Class to which the document belongs to.
- **docID:** Primary key of the instance to add.
- **newData:** Attribute values to add to the instance.

In this function, *newData* from a new instance identified by its *docID* from the class *collection* is added.

updateData (update)

```

updateData = async (collection, docID, updateData) => {
  try {
    const collectionRef = db.collection(collection);
    const docRef = collectionRef.doc(docID);

    await docRef.update(updateData);
    const updatedData = await docRef.get();

    if(updatedData.exists){
      const data ={
        id: updatedData.id,
        ...updatedData.data(),
      }
      return data
    } else {
      throw new Error(`Documento with ID ${docID} not found
after the update.`);
    }
  } catch (error) {
    console.error('Error updating data:', error);
    throw error;
  }
};

```

- **Collection:** Class to which the document belongs to.
- **docID:** Primary key of the instance to update.
- **updateData:** Attributes values to update to the instance.

In this function, *updateData* from an existing instance identified by its *docID* from the class *collection* is added.

deleteData (delete)

```
deleteData = async (collection, docID) => {
  try {
    const collectionRef = db.collection(collection);
    const docRef = collectionRef.doc(docID);
    await docRef.delete();
    return docRef.id;
  } catch (error) {
    console.error('Error deleting data:', error);
    throw error;
  }
};
```

- **Collection**: Class to which the document belongs to.

- **docID**: Primary key of the instance to delete.

In this function, an instance identified by its *docID* from the class *collection* is deleted.

4.1.3 API Endpoints

The API endpoints from the Router have been used for the classes *User*, *Club*, *Exercise*, *Team*, *Player*, *PlayerCharacteristics*, *PlayerStats*, *Training*, *Match* and *PlayerMatchStats*.

These endpoints serve as locations from where the frontend is going to ask for the information. Then, the API endpoints redirect these calls to the backend functions programmed, whose objective is to retrieve, add, update or delete the information needed.

The vast majority of classes use the 5 basic HTTP API Endpoints, which are the following²:

```
RouterServer.get('/users', CtrlUser.getAll);
RouterServer.get('/users/:id', CtrlUser.getById);
RouterServer.post('/users', CtrlUser.create);
RouterServer.patch('/users/:id', CtrlUser.update);
RouterServer.delete('/users/:id', CtrlUser.delete);
```

And some extra API Endpoints for certain controllers, as:

² It has been taken the User class as an example

verifyUser (User)

```
RouterServer.post('/verifyUser', CtrlUser.verifyUser);

static async verifyUser(req, res) {
    const result = validateUserLogin(req.body);

    if (!result.success) {
        const errorResponse =
result.error.errors.map(err => ({
            field: err.path[0],
            message: err.message
        }));

        return res.status(422).json({ errors:
errorResponse });
    }

    const { username, password } = req.body;

    try {
        const user = await ModelUser.getById({ username
});

        if (!user) {
            return res.status(404).json({ message: 'User
not found' });
        }

        if (user.password === password) {
            return res.json({ message: 'User verified'
});
        } else {
            return res.status(401).json({ message:
'Wrong password' });
        }
    } catch (error) {
        console.error("Error in login:", error.message);
        return res.status(500).json({ message: 'Internal
server error' });
    }
}
```

which validates the user login credentials in order to sign into the application with its account.

getAllRivalTeams (Team)

```
RouterServer.get('/users/:id/teams/:idT/rival_teams',
CtrlTeam.getAllRivalTeams);

static async getAllRivalTeams(req, res) {
    const id = req.params.id;
    const league = req.query.league;

    try {
        const documents = await ModelTeam.getAll();
        const filteredDocuments = documents.filter(doc
=> !doc.coach && doc.creator === id && doc.league === league);
        res.json(filteredDocuments);
    } catch (error) {
        console.log(error);
        res.status(500).json({ error: 'Internal Server
Error' });
    }
}
```

which gets all the rival teams from the same league of a certain team of the coach. This can be useful for knowing which teams the coach has to face throughout the season.

getByPlayer (PlayerCharacteristics)

```
RouterServer.get('/users/:id/teams/:idT/players/:idP/playercharac
teristics', CtrlPlayerCharacteristics.getByPlayer);

static async getByPlayer(req, res) {
    const idP = req.params.idP;

    // Player exists?
    try {
        const player = await ModelPlayer.getById({ id: idP
});
    } catch (error) {
        if (error.message === 'No existe el documento') {
            return res.status(404).json({ message: 'Player
not found' });
        }
        return res.status(500).json({ message: 'Internal
server error' });
    }
}
```

```

        try {
            const data = await
ModelPlayerCharacteristics.getByPlayer(idP);
            if (data) {
                res.json(data);
            } else {
                return res.status(404).json({ message:
'PlayerCharacteristics not found' });
            }
        } catch (error) {
            console.error("Error in getByPlayer:",
error.message);
            if (error.message === 'No existe el documento') {
                return res.status(404).json({ message:
'PlayerCharacteristics not found' });
            }
            return res.status(500).json({ message: 'Internal
server error' });
        }
    }
}

```

which gets the characteristics from one player. This can be useful to manage the player's personal information in a separate way. The same happens with the player stats.

getAllByUserAndTeam (Training)

```

RouterServer.get('/users/:id/teams/:idT/trainings',
CtrlTraining.getAllByUserAndTeam);

```

```

static async getAllByUserAndTeam(req, res) {
    const username = req.params.id;
    const idT = req.params.idT;
    try {
        const documents = await ModelTraining.getAll();
        const filteredDocuments = documents.filter(doc
=> doc.coach === username && doc.team === idT);
        res.json(filteredDocuments);
    } catch (error) {
        console.error("Error in getAllByUserAndTeam:",
error.message);
        if (error.message === 'Training not found') {
            return res.status(404).json({ message:
'Training not found' });
        }
    }
}

```

```

        return res.status(500).json({ message: 'Internal
server error' });
    }
}

```

which gets the trainings from a certain user and team. This is useful to know which trainings the user has planned for a certain team.

Usually, these extra functions are similar to the basic `getAll`, `getBy`, `create`, `update`, and `delete` ones but with different parameters or function behaviours.

4.2 Frontend

4.2.1 Frontend structure

For the frontend, the organization followed has been:

- **node_modules**: It contains all the installations and dependencies for the web application and the libraries used, like *react-router-dom*, *axios* and *web-vitals*, among others.
- **public**: All the needed assets used in the application.
- **src**: It contains all the executable code that will be used to implement the application web.
 1. **Components**: All the views that render the information sent by the backend and organize it in a visually attractive way.
 2. **Models**: All the classes that contain the attributes and getters representing the class models in code from.
 7. **CSS**: All the styles and designs from the component views.
 8. **Api**: All the functions used and sent from the frontend components to the endpoint definitions with the HTTP methods from the backend.

4.2.2 API Communication

The API communication from the frontend has been used for the classes *User*, *Club*, *Exercise*, *Team*, *Player*, *Training* and *Match* (plus *PlayerCharacteristics* and *PlayerStats*).

These functions serve as creations of API calls with the corresponding routes and bodies from where the frontend is going to ask for the information. Then, the API endpoints redirect these calls to the backend functions programmed, whose objective is to retrieve, add, update or delete the information needed.

Here are some examples of the 5 basic HTTP API calls creations used from the frontend³:

³ It has been taken the Club and User classes as an example.

getAll

```
const API_URL = "http://localhost:5000/clubs";

export async function getClubs() {
  try {
    const response = await fetch(API_URL, {
      method: "GET",
      headers: {
        Accept: "application/json",
        "Content-Type": "application/json",
      },
    });

    if (!response.ok) {
      const errorData = await response.json();
      throw new Error(errorData.error || "Error obtaining clubs");
    }

    return await response.json();
  } catch (error) {
    console.error("Error in getClubs:", error);
    throw error;
  }
}
```

getBy

```
import User from "../../models/ModelUser"

const API_URL = "http://localhost:5000/users";

export async function getUser(username) {
  try {

    const response = await fetch(`${API_URL}/${username}`, {
      method: "GET",
      headers: {
        Accept: "application/json",
        "Content-Type": "application/json",
      },
    });

  });
```

```

        if (!response.ok) {
            const errorData = await response.json();
            throw new Error(errorData.error || "Error obtaining
user");
        }

        const user = await response.json();
        const userP = {
            username: user.username,
            name: user.name,
            surname: user.surname,
            password: user.password
        }
        return new User(userP)

    } catch (error) {
        console.error("Error in getUser:", error);
        throw error;
    }
}

```

create

```

const API_URL = "http://localhost:5000/users";

export async function createUser(userData) {
    try {
        const response = await fetch(API_URL, {
            method: "POST",
            headers: {
                Accept: "application/json",
                "Content-Type": "application/json",
            },
            body: JSON.stringify(userData),
        });
        if (!response.ok) {
            let errorData;
            try {
                errorData = await response.json();
            } catch (error) {
                throw new Error('No se pudo procesar la respuesta
del servidor');
            }
        }
    }
}

```

```

        if (errorData.errors) {
            const errorMessages = errorData.errors.map(err =>
`${err.field}: ${err.message}`).join('\n');
            throw new Error(errorMessages);
        } else {
            throw new Error(errorData.message || "Error
creating user");
        }
    }

    return await response.json();
} catch (error) {
    console.error("Error in createUser:", error);
    throw error;
}
}

```

update

```

const API_URL = "http://localhost:5000/users/";

export async function updateUser(username, userData) {
    try {
        const response = await fetch(`${API_URL}${username}`, {
            method: "PATCH",
            headers: {
                Accept: "application/json",
                "Content-Type": "application/json",
            },
            body: JSON.stringify(userData),
        });
        if (!response.ok) {
            let errorData;
            try {
                errorData = await response.json();
            } catch (error) {
                throw new Error('No se pudo procesar la respuesta
del servidor');
            }

            if (errorData.errors) {
                const errorMessages = errorData.errors.map(err =>
`${err.field}: ${err.message}`).join('\n');
                throw new Error(errorMessages);
            }
        }
    }
}

```

```

        } else {
            throw new Error(errorData.message || "Error
updating user");
        }
    }

    return await response.json();
} catch (error) {
    console.error("Error in updateUser:", error);
    throw error;
}
}

```

delete

```

const API_URL = "http://localhost:5000/users";

export async function deleteUser(username) {
    try {

        const response = await fetch(`${API_URL}/${username}`, {
            method: "DELETE",
            headers: {
                Accept: "application/json",
                "Content-Type": "application/json",
            },
        });

        if (!response.ok) {
            let errorData;
            try {
                errorData = await response.json();
            } catch (error) {
                throw new Error('No se pudo procesar la
respuesta del servidor');
            }

            if (errorData.errors) {
                const errorMessages =
errorData.errors.map(err => `${err.field}:
${err.message}`).join('\n');
                throw new Error(errorMessages);
            } else {

```

```

        throw new Error(errorData.message || "Error
deleting user");
    }
}

return await response.json();

} catch (error) {
    console.error("Error in deleteUser:", error);
    throw error;
}
}

```

4.3 User interface

For the application to be successful, it had to be both intuitive and suitable for all types of coaches, mixing traditional aspects of football with innovative features, all while staying within the scope of training and professionalism.

Here are some interesting page layouts:

4.3.1 Main Page

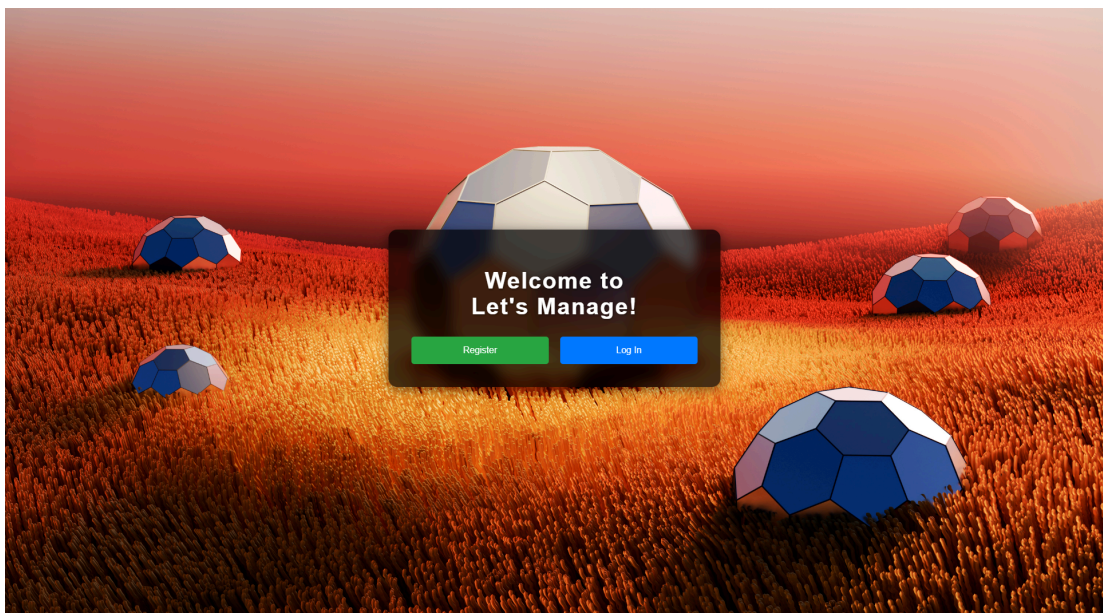


Figure 5. Main Page. Source: The author

4.3.2 Register

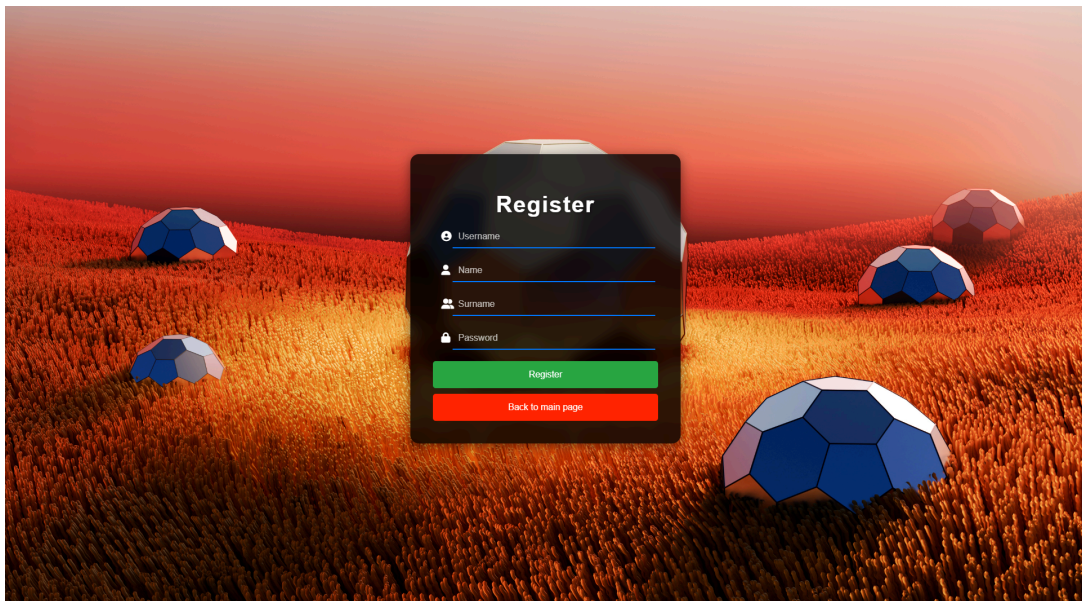


Figure 6. Register page. Source: The author

4.3.3 Log in

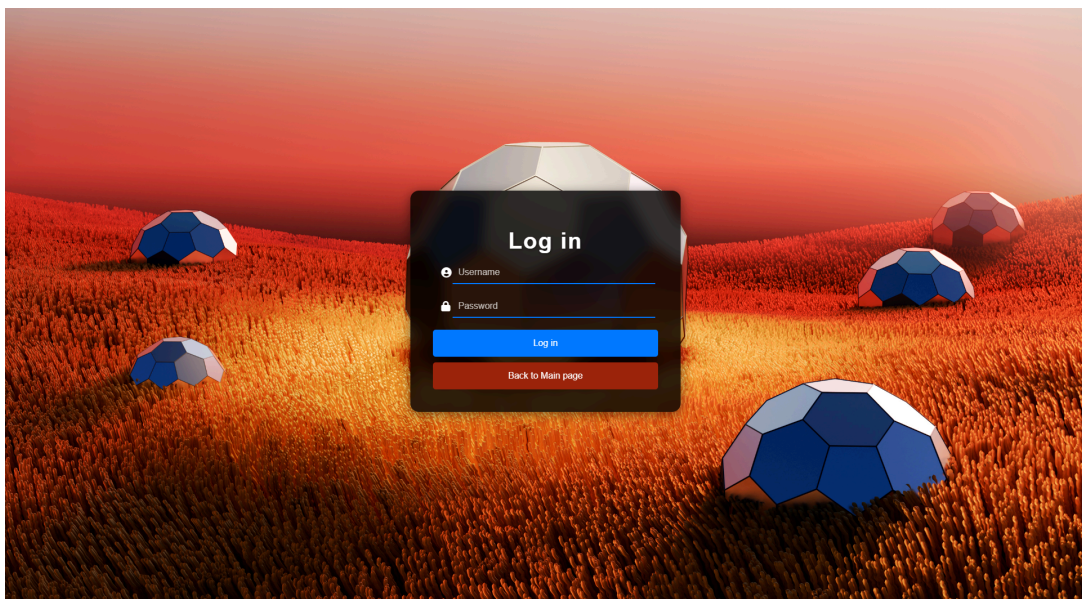


Figure 7. Login page. Source: The author

4.3.4 Home

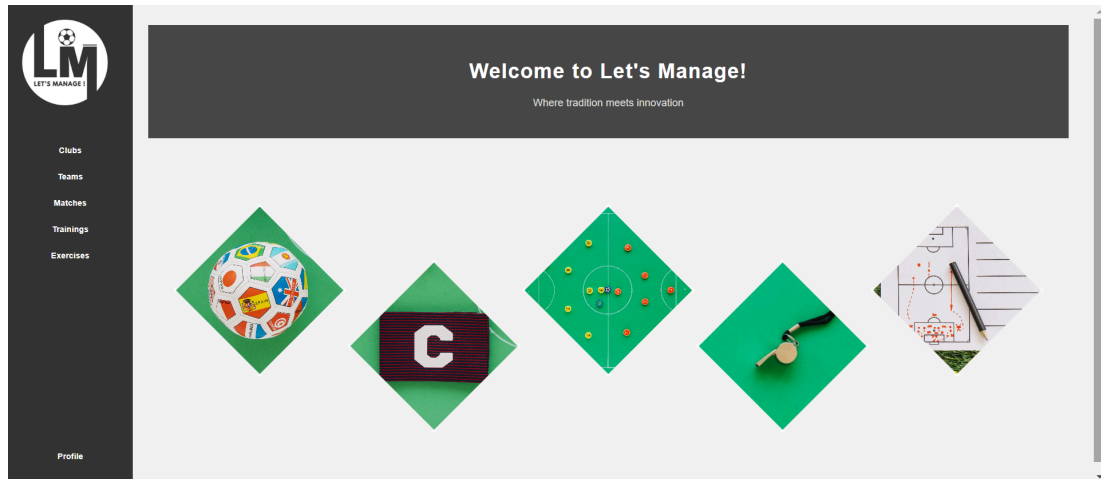


Figure 8. Home page. Source: The author

4.3.5 Profile

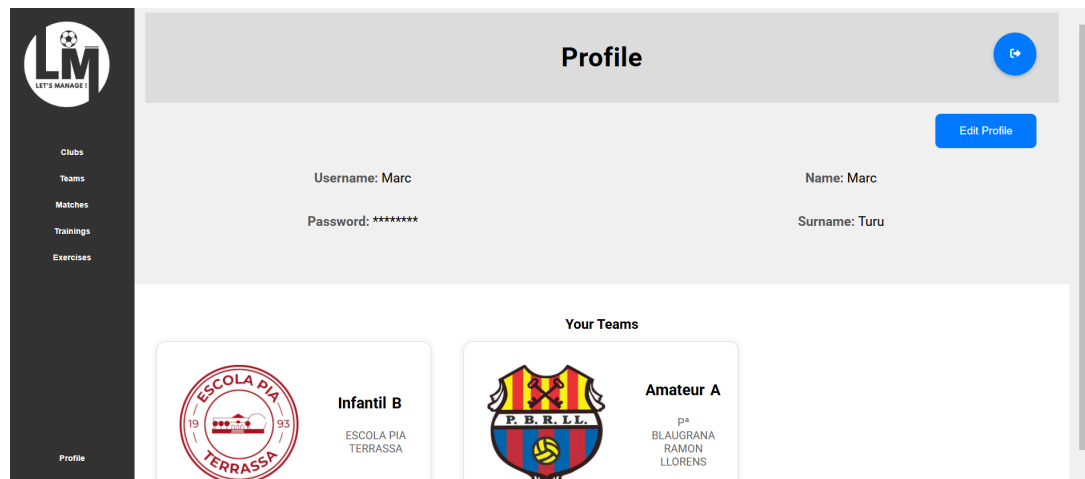


Figure 9. Profile page. Source: The author

4.3.6 Clubs

Clubs			
Photo	Name	City	Location
	AGORA SANT CUGAT SPORTS CLUB	Sant Cugat	Carrer Ferrer i Guardia, nº 1
	CAN BOADA, C.F.	Terrassa	Cami de Can Boada
	CAN PARELLADA, C.D.	Terrassa	Carrer Àmerica, nº 8
	CAN TRIAS, U.D.	Terrassa	Carrer Isaac Albeniz, nº 33
	CASTELLBISBAL, U.E.	Castellbisbal	Carrer Major, nº 78
	D'ARAN CLUB ASSOCIACIÓ CULTURAL	Mira-sol	Avda. Bilbao, nº 97-103

Figure 10. Clubs page. Source: The author

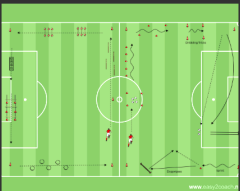
4.3.7 Exercise(s)



- Clubs
- Teams
- Matches
- Trainings
- Exercises
- Profile

Exercises

Type: All
Only personalized:



Conditioning 1

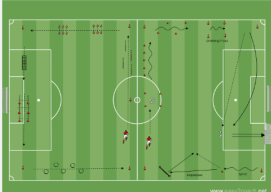
Duration: 10 min + 10 min
Personalized: No

Figure 11. Exercises page. Source: The author



- Clubs
- Teams
- Matches
- Trainings
- Exercises
- Profile

Conditioning 1



Description:
There are 2 teams which make their way through the different phases of the exercise until they reach the starting cone. Repeat for 10 minutes and change sides.

Duration:
10 min + 10 min

Material:

- Balls
- Minigoals
- Cones
- Ladders
- Rings

Personalized:
No

Figure 12. Exercise page. Source: The author

4.3.8 Team(s)

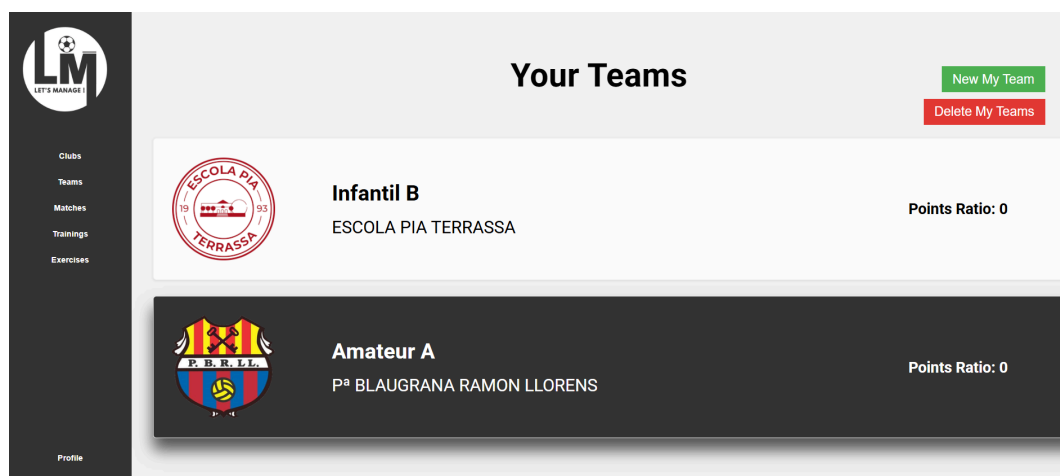


Figure 13. Teams page. Source: The author

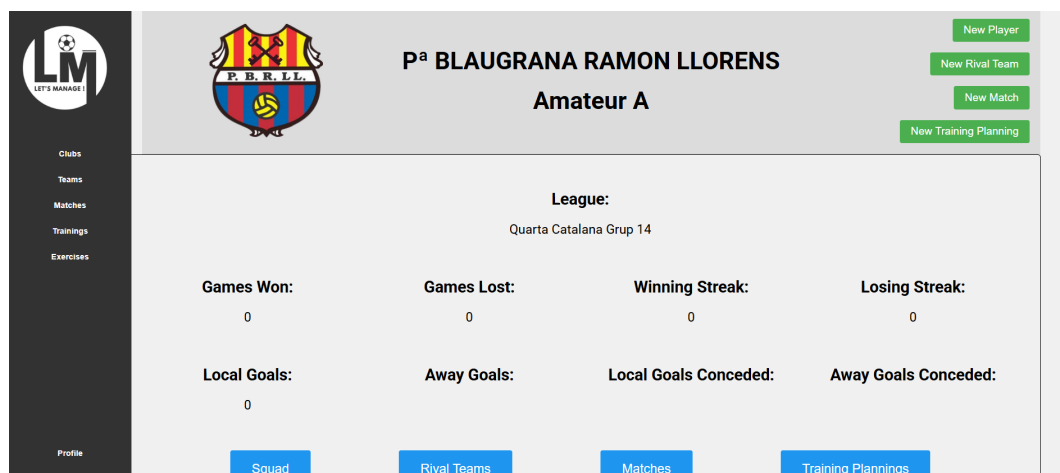


Figure 14. Team page. Source: The author

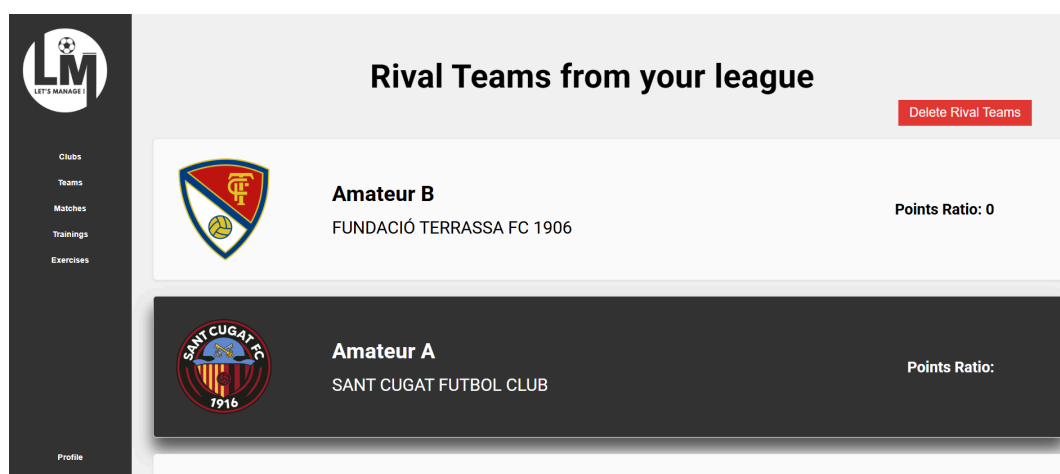


Figure 15. Rival teams page. Source: The author



- Clubs
- Teams
- Matches
- Trainings
- Exercises
- Profile



SANT CUGAT FUTBOL CLUB

Amateur A

Edit

League:	Strengths:			Weaknesses:
Quarta Catalana Grup 14	Defense, counter-attack, keeper			Core, shots
Games Won:	Games Lost:	Winning Streak:	Losing Streak:	
5	2	3	0	
Local Goals:	Away Goals:	Local Goals Conceded:	Away Goals Conceded:	
7	5	0	3	

Figure 16. Rival team page. Source: The author

4.3.9 Player(s)

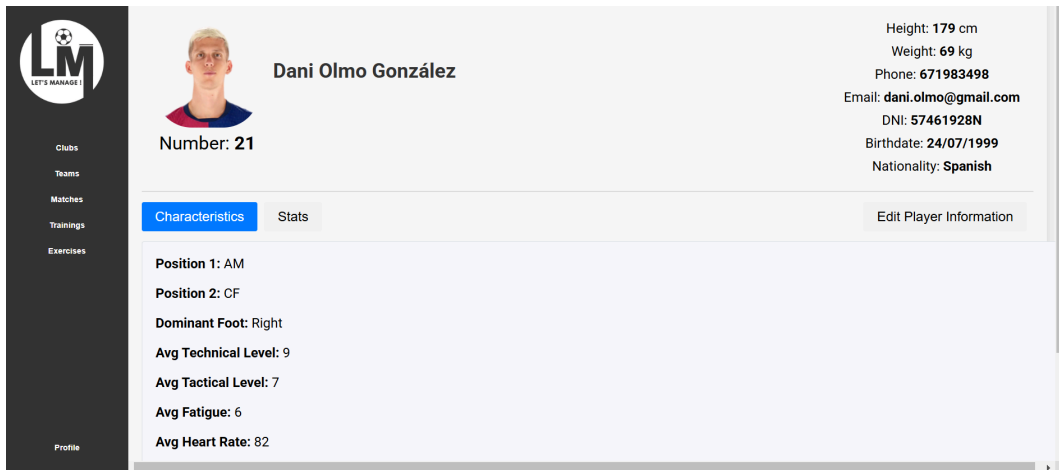


- Clubs
- Teams
- Matches
- Trainings
- Exercises
- Profile

Players

 Alejandro Balde Roca 2	 Ander Astralaga Romeu 13	 Andreas Christensen Khris 3
 Ansu Fati Drisi 10	 Dani Olmo González 21	 Eric García Rodríguez 5
 Fermín López Aranda 14	 Ferran Torres Sevilla 12	 Frenkie DeJong VanDijk 15
 Gerard Martín Rovira 24	 Héctor Fort Sotelo 19	 Iñaki Peña Aymerich 30

Figure 17. Players page. Source: The author



LM
LET'S MANAGE!

Clubs
Teams
Matches
Trainings
Exercises

Profile


Dani Olmo González
Number: 21

Height: 179 cm
Weight: 69 kg
Phone: 671983498
Email: dani.olmo@gmail.com
DNI: 57461928N
Birthdate: 24/07/1999
Nationality: Spanish

Characteristics Stats Edit Player Information

Position 1: AM
Position 2: CF
Dominant Foot: Right
Avg Technical Level: 9
Avg Tactical Level: 7
Avg Fatigue: 6
Avg Heart Rate: 82

Figure 18. Player page. Source: The author

4.3.10 Match(es)



LM
LET'S MANAGE!

Clubs
Teams
Matches
Trainings
Exercises

Profile

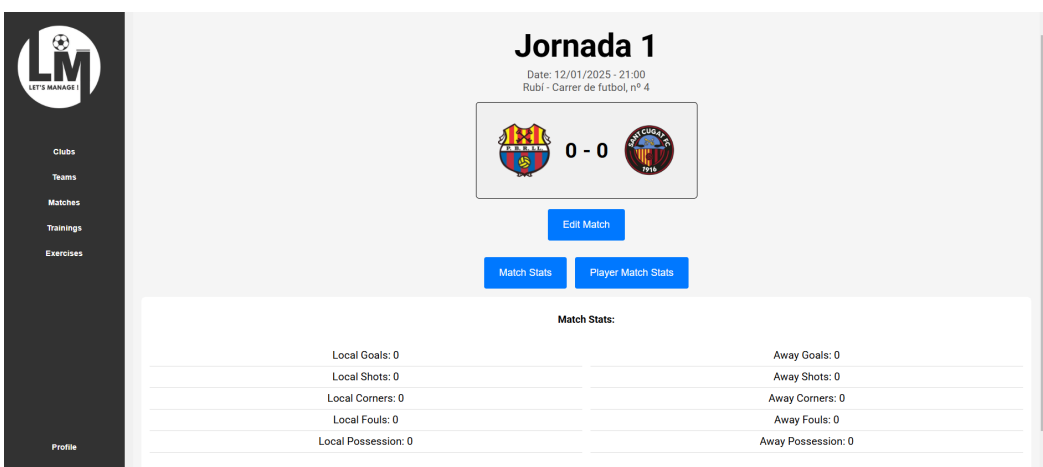

Matches from P^a BLAUGRANA RAMON LLORENS Amateur A

Delete Match

 **Jornada 1 VS SANT CUGAT FUTBOL CLUB Amateur A** **12/01/2025 - 21:00**
Rubí - Carrer de futbol, nº 4

 **Jornada 2 VS FUNDACIÓ TERRASSA FC 1906 Amateur B** **19/01/2025 - 21:00**
Rubí - Carrer de futbol, nº 4

Figure 19. Matches page. Source: The author



LM
LET'S MANAGE!

Clubs
Teams
Matches
Trainings
Exercises

Profile

Jornada 1
Date: 12/01/2025 - 21:00
Rubí - Carrer de futbol, nº 4

 **0 - 0** 

Edit Match

Match Stats Player Match Stats

Match Stats:

Local Goals: 0	Away Goals: 0
Local Shots: 0	Away Shots: 0
Local Corners: 0	Away Corners: 0
Local Fouls: 0	Away Fouls: 0
Local Possession: 0	Away Possession: 0

Figure 20. Match page. Source: The author

4.3.11 Training(s)

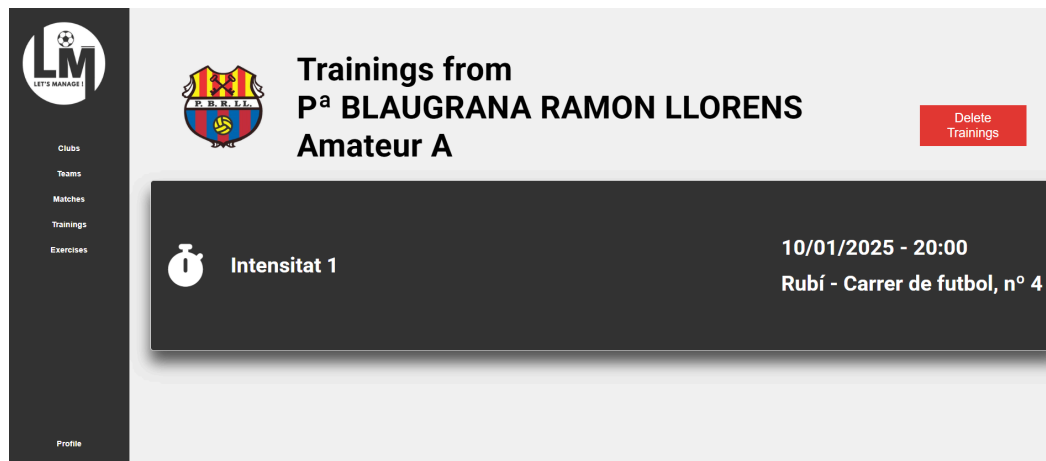


Figure 21. Trainings page. Source: The author

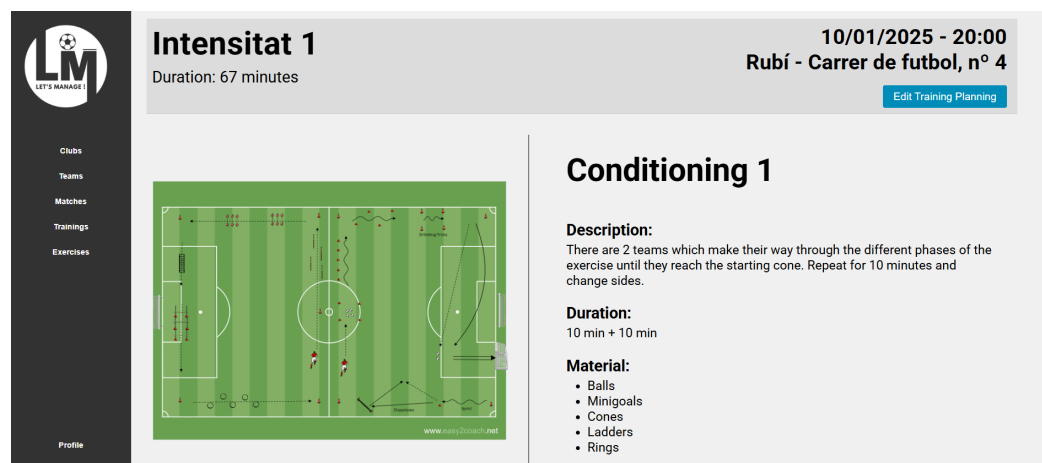


Figure 22. Training page. Source: The author

5. Testing

5.1 API endpoint tests

The tests have been organized to validate all the API endpoint calls.

To do so, the following files have been created: *apiClub.http*, *apiExercise.http*, *apiMatch.http*, *apiPlayer.http*, *apiPlayercharcateristics.http*, *apiPlayerMatchStats.http*, *apiPlayerStats.http*, *apiTeam.http*, *apiTraining.http* and *apiUser.http*.

Basically, from every controller class, an apiX.http file has been created in order to evaluate every API call that they will receive. It is intended to serve as a method of control to evaluate all the possible scenarios from every API endpoint and confirm their correct usage.

In the following sections, all the function tests validations are presented⁴:

5.1.1 GetAll (Club)

```
GET http://localhost:5000/clubs
```

Parameters: -

Body: -

Response: [{ "id": "AGORA SANT CUGAT SPORTS CLUB", "photo": "/assets/Clubs/Agora.png", "city": "Sant Cugat", "location": "Carrer Ferrer i Guardia, nº 1" }, { ... }]

5.1.2 GetById (User)

```
GET http://localhost:5000/users/Marc
```

Parameters: id (string)

Body: -

Response: { "id": "Marc", "username": "Marc", "name": "Marc", "surname": "Turu", "password": "12345678" }

5.1.3 Create (User)

```
POST http://localhost:5000/users
```

Parameters: -

Body: username (string), name (string), surname (string), password (string)

Response: { "username": "Marc", "name": "Marc", "surname": "Marc", "password": "12345678" }

⁴ If the test is repeated in different classes, it is only included once.

5.1.4 Update (User)

```
PATCH http://localhost:5000/users/Marc
```

Parameters: id (string)

Body: password (string) . (*For example*)

Response: { "username": "Marc", "name": "Marc", "surname": "Marc",
"password": "newPassword" }

5.1.5 Delete (User)

```
DELETE http://localhost:5000/users/Marc
```

Parameters: id (string)

Body: -

Response: { "message": "User Marc deleted" }

5.1.6 Verify user credentials

```
POST https://localhost:5000/verifyUser
```

Parameters: -

Body: username (string) , password (string)

Response: { "message": "User verified" }

5.1.7 GetAllByType (Exercise)

```
GET http://localhost:5000/exercises?type=duels
```

Parameters: -

Body: -

Query: type=duels (*For example*)

Response: [{ "id": "Duels 1", "description": "There are 2 teams.
The team A attacks the 2 mini goals in a 4v3 (with superiority).
Once the action ahs finished, the team B attacks the other two
minigoals in a 4v3 (with superiority). This action is repeated",
"material": ["balls", "minigoals", "cones", "vests"],
"personalized": false, "type": "duels", "photo
"assets/Exercises/duels1.png", "duration": "7 min + 7 min" }, {
... }]

5.1.8 GetAllByPersonalized (Exercise)

```
GET http://localhost:5000/exercises?personalized=true
```

Parameters: -

Body: -

Query: personalized=true (*For example*)

Response: [{ "id": "Conditioning 2", "description": "Sprint route between cones. 10 meters + 30 meters + 10 meters", "personalized": true, "type": "conditioning", "photo": "assets/Exercises/conditioning2.png", "duration": "5 min + 5 min + 5 min", "material": ["Cones"] }, { ... }]

5.1.9 GetAllByTypeAndPersonalized (Exercise)

```
GET http://localhost:5000/exercises?type=duels&personalized=true
```

Parameters: -

Body: -

Query: type=duels & personalized=true (*For example*)

Response: [{ "id": "Duels 7", "description": "There are 2 teams. Team A recieves the ball from one teammate and starts the 1v1 situation, trying to score in a minigoal after going thgrough 2 cones. If team B recovers the ball, they have to score in the other minigoal. The action is repeated and the sides are changed.", "type": "duels", "material": ["balls", "minigoals", "cones", "vests"], "personalized": true, "photo": "assets/Exercises/duels7.png", "duration": "7 min + 7 min" }]

5.1.10 GetAllByTeam (Player)

```
GET http://localhost:5000/users/Marc/teams/Pa BLAUGRANA RAMON LLORENS_amateur_a_Marc/players
```

Parameters: -

Body: -

Response: [{ "id": "marc_turu_roca", "name": "Marc", "surname": "Turu", "surname2": "Roca", "height": 180, "weight": 70, "phone": "618043499", "mail": "marc.turu@gmail.com", "DNI": "54755698S", "birthdate": "25/04/2002", "nationality": "Spanish", "number": 6, "team": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc", "photo": "/assets/voidPFP.jpg" } { ... }]

5.1.11 GetAllByUser (Match)

```
GET http://localhost:5000/users/Marc/matches
```

Parameters: -

Body: -

Response: [{ "id": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc_Jornada 4", "name": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc_Jornada 4", "myTeam": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc", "rivalTeam": "CAN PARELLADA, C.D._amateur_a_Marc", "city": "Rubí", "address": "Carrer de futbol, nº 4", "date": "15/12/2024", "hour": "22:45", "imLocal": true, "finished": false, "formation": 433, "starters": ["marc_turu_roca", "pere_turu_roca", "arnau_turu_roca", "oriol_turu_roca", "alex_turu_roca", "didac_turu_roca", "aleix_turu_roca", "sergi_turu_roca", "armand_turu_roca", "gerard_turu_roca", "carles_turu_roca"], "calledups": ["marc_turu_roca", "pere_turu_roca", "arnau_turu_roca", "oriol_turu_roca", "alex_turu_roca", "didac_turu_roca", "aleix_turu_roca", "sergi_turu_roca", "armand_turu_roca", "gerard_turu_roca", "carles_turu_roca", "xavier_turu_roca", "roger_turu_roca", "amadeu_turu_roca", "aniol_turu_roca"], "myGoals": 0, "rivalGoals": 0, "myPossession": 0, "rivalPossession": 0, "myShots": 0, "rivalShots": 0, "myFouls": 0, "rivalFouls": 0, "myCorners": 0, "rivalCorners": 0 }, { ... }]

5.1.12 GetAllByUserAndTeam (Match)

```
GET http://localhost:5000/users/Marc/teams/Pa BLAUGRANA RAMON LLORENS_amateur_a_Marc/matches
```

Parameters: -

Body: -

Response: [{ "id": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc_Jornada 4", "name": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc_Jornada 4", "myTeam": "P^a BLAUGRANA RAMON LLORENS_amateur_a_Marc", "rivalTeam": "CAN PARELLADA, C.D._amateur_a_Marc", "city": "Rubí", "address": "Carrer de futbol, nº 4", "date": "15/12/2024", "hour": "22:45", "imLocal": true, "finished": false, "formation": 433, "starters": ["marc_turu_roca", "pere_turu_roca", "arnau_turu_roca", "oriol_turu_roca", "alex_turu_roca", "didac_turu_roca", "aleix_turu_roca", "sergi_turu_roca", "armand_turu_roca", "gerard_turu_roca", "carles_turu_roca"], "calledups": ["marc_turu_roca", "pere_turu_roca", "arnau_turu_roca", "oriol_turu_roca", "alex_turu_roca", "didac_turu_roca", "aleix_turu_roca", "sergi_turu_roca", "armand_turu_roca", "gerard_turu_roca", "carles_turu_roca", "xavier_turu_roca",

```
"roger_turu_roca", "amadeu_turu_roca", "aniol_turu_roca"],  
"myGoals": 0, "rivalGoals": 0, "myPossession": 0,  
"rivalPossession": 0, "myShots": 0, "rivalShots": 0, "myFouls":  
0, "rivalFouls": 0, "myCorners": 0, "rivalCorners": 0 }, { ... }]
```

5.2 Test scenarios

After presenting all the API tests, it is convenient to analyze the response codes shown when executing them, treating firstly the positive cases and secondly the negative ones.

5.2.1 Positive scenarios

200 OK: Succes.

```
HTTP/1.1 200 OK  
X-Powered-By: Express  
Access-Control-Allow-Origin: *  
Content-Type: application/json; charset=utf-8  
Content-Length: 4534  
ETag: W/"11b6-6xqVCOxxfYTS6xqjU/pfIsR0F0U"  
Date: Sun, 05 Jan 2025 20:35:19 GMT  
Connection: close  
  
GET http://localhost:5000/clubs
```

Figure 23. 200 OK Response

201 Created: Creation.

```
HTTP/1.1 201 Created  
X-Powered-By: Express  
Access-Control-Allow-Origin: *  
Content-Type: application/json; charset=utf-8  
Content-Length: 73  
ETag: W/"49-kHlfnaSMjHscPcjmvZGj1heocMM"  
Date: Sun, 05 Jan 2025 20:52:48 GMT  
Connection: close  
  
POST http://localhost:5000/users  
Content-Type: application/json  
  
{  
  "username": "CoachMarc",  
  "name": "Marc",  
  "surname": "Turu",  
  "password": "12345678"  
}
```

Figure 24. 201 Created Response

5.2.2 Negative scenarios

400 Bad Request: Invalid data.

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 1111
Date: Mon, 06 Jan 2025 05:23:52 GMT
Connection: close

POST http://localhost:5000/users
Content-Type: application/json

{
  "username": Marc,
  "name": "Marc",
  "surname": "Turu",
  "password": "12345678"
}
```

Figure 25. 400 Bad Request Response

401 Unauthorized: Wrong password.

```
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 28
ETag: W/"1c-1DgWgAxsia6W2YD8mP0Sbp4yZkI"
Date: Mon, 06 Jan 2025 10:28:45 GMT
Connection: close

POST http://localhost:5000/verifyUser
Content-Type: application/json

{
  "username": "Marc",
  "password": "BadPassword"
}
```

Figure 26. 401 Unauthorized Response

404 Not Found: Not exists.

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 144
Date: Mon, 06 Jan 2025 05:22:30 GMT
Connection: close

GET http://localhost:5000/users/Nobody
```

Figure 27. 404 Not Found Response

409 Conflict: Already exists.

```
HTTP/1.1 409 Conflict
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 52
ETag: W/"34-NeCRSoHz7oheUCiJUpZCZjGwxXo"
Date: Mon, 06 Jan 2025 10:37:02 GMT
Connection: close

POST http://localhost:5000/users
Content-Type: application/json

{
  "username": "Marc",
  "name": "Marc",
  "surname": "Turu",
  "password": "12345678"
}
```

Figure 28. 409 Conflict Response

422 Unprocessable Entity: Format validation error.

```
HTTP/1.1 422 Unprocessable Entity
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 78
ETag: W/"4e-tcWUwg9HvARDqLbU9kAPAdhk11o"
Date: Mon, 06 Jan 2025 05:31:13 GMT
Connection: close

POST http://localhost:5000/users
Content-Type: application/json

{
  "username": "Marc",
  "name": "Marc",
  "surname": "Turu",
  "password": "1"
}
```

Figure 29. 422 Unprocessable Entity Response

500 Internal Server Error: Unhandled exception.

```
HTTP/1.1 500 Internal Server Error
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 33
ETag: W/"21-Fau8GdrOCoyGNNH/IiTxy2DuMu0"
Date: Mon, 06 Jan 2025 10:42:15 GMT
Connection: close

GET http://localhost:5000/users
```

Figure 30. 500 Internal Server Error Response

As shown, before confirming the finalization of the implementation of a functionality, loads of tests (on positive, negative and extreme cases) have been done. This is why, for each API Endpoint, all the possible scenarios have been studied and all the error appearances have been taken into account and properly treated.

All the classes have their own http tester, so the coverage is huge and reliable.

6. Project management

6.1 Methodology and rigor

There are lots of methodologies on how software projects can be based on [5]. From ones grounded on daily basis and weekly updates to others way more passive and regulated from start to finish.

Considering the type of project and the previous experience with it, the methodology used will be *Agile* and the Scrum management [6]. This methodology divides the work in sprint iterations of different time depending on the load of work from each division (the time plannings will be discussed in the following chapters).

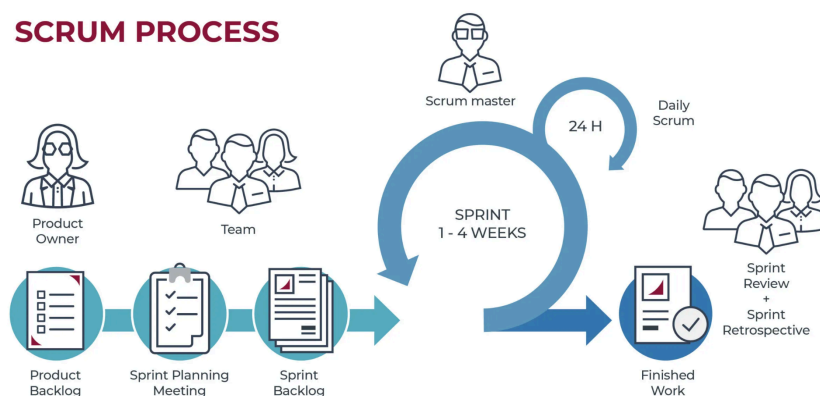


Figure 31. Scrum basic process. Source: [7]

Although Scrum is a project management methodology that adapts well with teams and I'm working alone, I decided to use this one because the fact that having to be the **Scrum Master** and doing **Sprint Plannings** at the beginning of every sprint to state the objectives of it, **Sprint Reviews** in which I will go over the iterations done in that sprint and **Sprint Retrospectives** to analyze what has been well done or bad done along the sprint, every time will be a good way of keeping me updated with the work completed and with possible problems that could appear.

Moreover, this methodology will allow me to have better communication with the *TFG* director, as we could establish periodic and retrospective meetings.

As I want to keep track of the tasks I need to implement, I will use Taiga to do so.

This tool will allow me to divide all the procedures in epics, which in turn will split itself in user stories stored in the **Backlog**. The user stories will be big functionalities that the application will need to complete. Thus, these user stories will be divided into little tasks that

will have some status in order to know if they are New, In Progress, Ready for test, Closed or Needs Info.

This classification will let me benefit from another methodology: *Kanban* [8].

Help

Q

≡

△

↶

≡

📖

👤

John Doe

FILTERS

Subject or reference

Q

Advanced

TYPE

SEVERITY

PRIORITIES

STATUS

TAGS

ASSIGNED TO

ROLE

CREATED BY

ISSUES

THE PRINCESS BRIDE

Type	Severity	Priority	Votes	Subject	Status	Modified	Assigned to
▲ 1				#53 Rodents of unusual size (R.O.U.S.)	In progress	09 Oct 2019	The Man L...
▲ 0				#52 Lightning sand	In progress	09 Oct 2019	Buttercup...
▲ 0				#51 Flame spurts	In progress	20 Jul 2020	The Man L...
▲ 0				#50 Life is pain	New	09 Oct 2019	The Man L...
▲ 0				#49 Not a lot of money in revenge	New	17 Aug 2020	Wigo Mo...
▲ 0				#48 He didn't fall!	In progress	09 Oct 2019	Vizzini
▲ 0				#47 Unemployed in Greenland	Closed	09 Oct 2019	Fezzik
▲ 0				#46 Friendless, brainless, helpless, ho...	Ready to test	09 Oct 2019	Fezzik
▲ 0				#45 Princess Buttercup does not love ...	Needs Info	09 Oct 2019	Grandfat...
▲ 0				#44 Not enough money	Closed	09 Oct 2019	The Man L...
▲ 0				#43 Where's the sports in the book?	Ready to test	09 Oct 2019	Grandfat...
▲ 0				#42 The boy is sick in bed	In progress	09 Oct 2019	Grandfat...
▲ 0				#10 Dropping the sword when left-ha...	Closed	09 Oct 2019	Wigo Mo...
▲ 3				#9 Excessive exposure to locale	Postponed	09 Oct 2019	The Man L...

Get Started

Figure 32. Taiga project management example. Source: [9]

As far as version control is concerned, Github will be used.

Owing to its easy usage, convenient distribution and failure recovery, Github is a very good option when talking about code management. In it, there will be a repository that will contain the code of the application with the pertinent *src*, *configuration* and *test* folders for each of the *frontend* and *backend* subdivisions.

To elaborate the code, I will use the *Gitflow* methodology, where a *Master (main)* branch will contain the official code with no errors and ready to deploy. In a lower level, the *Develop* branch will be the one that will integrate all the functionalities from the application. And in a lower level there will be the *user stories/tasks* branches in which the different functionalities will be implemented independently in order to prevent errors.

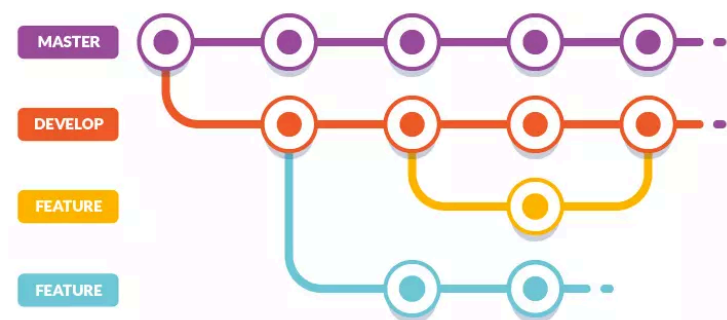


Figure 33. Github branches distribution. Source: [10]

6.2 Initial tasks management

This project began on the 18th September, 2024 with the first class of GEP and is expected to end (at the latest) on the 24th January, 2025 with the thesis defense in front of the committee. There are 129 days between these dates. Due to this, and taking into account that the TFG is worth 18 credits ECTS (each credit represents 30 hours) I estimate to work 4 hours a day more or less. However, if I wanted to rest on the weekends, I would have to work a bit more than 6 hours a day 5 days a week.

If the GEP management is divided from the project itself, and knowing that GEP is worth 3 credits, 90 hours from the 540 total would need to be allocated for GEP. Therefore, the remaining 450 would have to be allocated for the other tasks of the project.

Keeping in mind that this project has been developed using the Scrum methodology, with what this entails (defining each task at the beginning of the Sprint Planning with the Sprint Backlog and changing them throughout the development of the project), the tasks have been divided in 2 groups: Project management and Project development.

6.2.1 Project management

These tasks refer to documentation, planning and communications. They include the 90 hours from GEP plus 100 extra documentation hours:

- **PM1 → Contextualize and define scope (20 hours):** Documentation of the first delivery. Definition of the scope of the project in the context of its study.
Dependencies: - .
- **PM2 → Schedule temporal planning (15 hours):** Documentation of the second delivery. Definition of the phases of the project, resources and risks management.
Dependencies: PM1.
- **PM3 → Plan budget and sustainability (15 hours):** Documentation of the third delivery. Analysis of the economic plan and sustainability of the project.
Dependencies: PM2.
- **PM4 → Integrate the final document (20 hours):** Integration of the documents from PM1, PM2 and PM3 with the relevant changes. **Dependencies:** PM3.
- **PM5 → Meetings (20 hours):** Communications with the TFG supervisor during the elaboration of the project to control its evolution. **Dependencies:** - .
- **PM6 → Documenting and noting during the process:** Documentation during the project to keep track of it and adding extra notes. **Dependencies:** - .

- **PM7 → Documenting the final report (70 hours):** Writing of the whole project final report. **Dependencies:** - .
- **PM8 → Preparing the final defense (30 hours):** Elaboration of the final oral exposition and its visual elements in order to present it in front of the committee. **Dependencies:** PM7.

6.2.2 Project development

These tasks refer to definition, development and implementation of the app functionalities. They include 325 hours:

Inception

- **I1 → Prepare the environment setup (8 hours):** Install the different technologies to develop the app and work with it. **Dependencies:** - .
- **I2 → Detail the entire product backlog (7 hours):** Present all the user stories and their important information. **Dependencies:** - .
- **I3 → Elaborate the UML diagram and its DDBB (10 hours):** Detail all the objects, attributes and relations of the classes. **Dependencies:** I1.

User management

- **UM1 → Identify as a user (25 hours):** Creation of all the interfaces and backend code to log in as a new user and log out. **Dependencies:** Inception.
- **UM2 → Edit user profile (15 hours):** Creation of all the interfaces and backend code to edit the user preferences. **Dependencies:** UM1.

Team management

- **TeM1 → Create and edit team (30 hours):** Creation of all the interfaces and backend code to create new teams and edit information from them and their season. **Dependencies:** User management.
- **TeM2 → Edit and review player data (35 hours):** Creation of all the interfaces and backend code to add player useful information. **Dependencies:** TeM1.

Training management

- **TrM1 → Prepare training plannings (30 hours):** Creation of all the interfaces and backend code to elaborate training activities. **Dependencies:** Team management.
- **TrM2 → Prepare personalized training (40 hours):** Creation of all the interfaces and backend code to elaborate personalized training exercises. **Dependencies:** TrM1.

Match management

- **MM1 → Present match strategies (40 hours):** Creation of all the interfaces and backend code to create match lineups and strategies. **Dependencies:** Training management.

- **MM2 → Update post-match statistics (30 hours):** Creation of all the interfaces and backend code to allow coaches to add post-match information. **Dependencies:** MM1.

Rewards management

- **RM1 → Elaborate rewards system (25 hours):** Creation of all the interfaces and backend code to create and obtain rewards from statistics. **Dependencies:** Match management.

Healthy management

- **HM1 → Offer healthy plans (30 hours):** Creation of all the interfaces and backend code to design healthy programs for players. **Dependencies:** Rewards management.

Extra management

I left the remaining 25 hours to be invested in Sprint Planning (and Sprint Reviews), solution to risks and other important aspects as testing and documentation extras.

Therefore, bearing in mind the 90 hours from **GEP**, the 100 hours from **Documentation**, the 325 hours from **Project development** and the 25 hours from **Extras**, the 540 hours of **TFG project** would be reached.

6.2.3 Resources

Human Resources

ID	Resource	Person
HRPM	Project Manager	Marc Turu
HRSA	Software architect	Marc Turu
HRSD	Software developer	Marc Turu
HRT	Tester	Marc Turu
HRSTM	Scrum Team Member	Marc Turu
HRTR	Thesis Writer	Marc Turu
HRTS	Thesis Supervisor	Xavier Oriol Hilari
HRGT	GEP Tutor	Xavier Morales Sorolla

Table 3. Human resources involved in the project with ID, Resource & Person. Source: The Author.

Material Resources

ID	Resource	Why
MRWE	Work environment (Room, chair and table)	
MRPC	Computer (i5, 16GM RAM, GEFORCE RTX)	
MRMon	Monitor (LG ULTRAGEAR)	
MRK	Keyboard (Nemskill)	
MRMou	Mouse (G203 LIGHTSYNC)	
MRGP	GanttProject [11]	Elaborate the Gantt diagram
MRZ	Zotero [12]	Quote references
MRVSC	Visual Studio Code [13]	Framework used
MRG	Github [14]	Control versions
MRT	Taiga [15]	Manage the project
MRA&R	Atenea & Racó [16]	Find information from the project development
MRG	Gmail [17]	Communicate with parties involved in the project
MRGM	Google Meet [18]	Communicate with the thesis tutor
MRGC	Google Chrome [19]	Browse through the internet

Table 4. Material resources used in the project with ID, Resource & Why. Source: The Author.

6.2.4 Tasks summary

In the following table, a brief summary of the tasks of the project can be seen.

It shows the sprint they belong to (taking into account that I will probably work in 5 sprints of 2 weeks each from the end of the Inception to the end of December). However, the sprints where the tasks belong to may change depending on the development of each of them.

Code	Task	Workload	Dependencies	Resources	Sprint
	Project management	190 h		Basic Material Resources	
PM1	Contextualize and define scope	20 h		HRTR, HRTS, HRGT, MRZ, MRA&R	
PM2	Schedule temporal planning	15 h	PM1	HRTR, HRTS, HRGT, MRZ, MRA&R, MRGP	

PM3	Plan budget and sustainability	15 h	PM2	HRTR, HRTS, HRGT, MRZ, MRS&R	
PM4	Integrate the final document	20 h	PM3	HRTR, HRTS, HRGT, MRZ, MRA&R, MRGP	
PM5	Meetings	20 h		HRPM, HRTS, MRG, MRGM	
PM6	Documenting and noting during the process			HRTR, HRTS, MRZ	
PM7	Documenting the final report	70 h		HRTR, HRTS, MRGP, MRZ	
PM8	Preparing the final defense	30 h	PM7	HRTR, HRTS	
	Project development	325 h		Basic Material Resources	
I1	Prepare the environment setup	8 h		MRVSC, MRG	
I2	Detail the entire product backlog	7 h		HRPM, MRVSC, MRG	
I3	Elaborate the UML diagram and its DDBB	10 h	I1	HRPM, HRSA, MRVSC, MRG	
UM1	Identify as a user	25 h	Inception	HRSD, MRVSC, MRG, MRT	1
UM2	Edit user profile	15 h	UM1	HRSD, MRVSC, MRG, MRT	1
TeM1	Create and edit team	30 h	User management	HRSD, MRVSC, MRG, MRT	1
TeM2	Edit and review player data	35 h	TeM1	HRSD, MRVSC, MRG, MRT	2
TrM1	Prepare training plannings	30 h	Team management	HRSD, MRVSC, MRG, MRT	2
TrM2	Prepare personalized trainings	40 h	TrM1	HRSD, MRVSC, MRG, MRT	3
MM1	Present match strategies	40 h	Training management	HRSD, MRVSC, MRG, MRT	3
MM2	Update post-match statistics	30 h	MM1	HRSD, MRVSC, MRG, MRT	4
RM1	Elaborate rewards system	25 h	Match management	HRSD, MRVSC, MRG, MRT	4
HM1	Offer healthy plans	30 h	Rewards management	HRSD, MRVSC, MRG, MRT	5
	Extra management	25 h		HRPM, HRT, HRSTM, HRTS	1-5

Table 5. Task summary with Code, Task, Workload, Dependencies, Resources and Sprint. Source: The Author.

6.2.5 Gantt chart

The following Figure shows the Gantt chart divided by groups and tasks. In spite of GanttProject limitations, I have already included the relation between Sprints and activity tasks in Table 3.

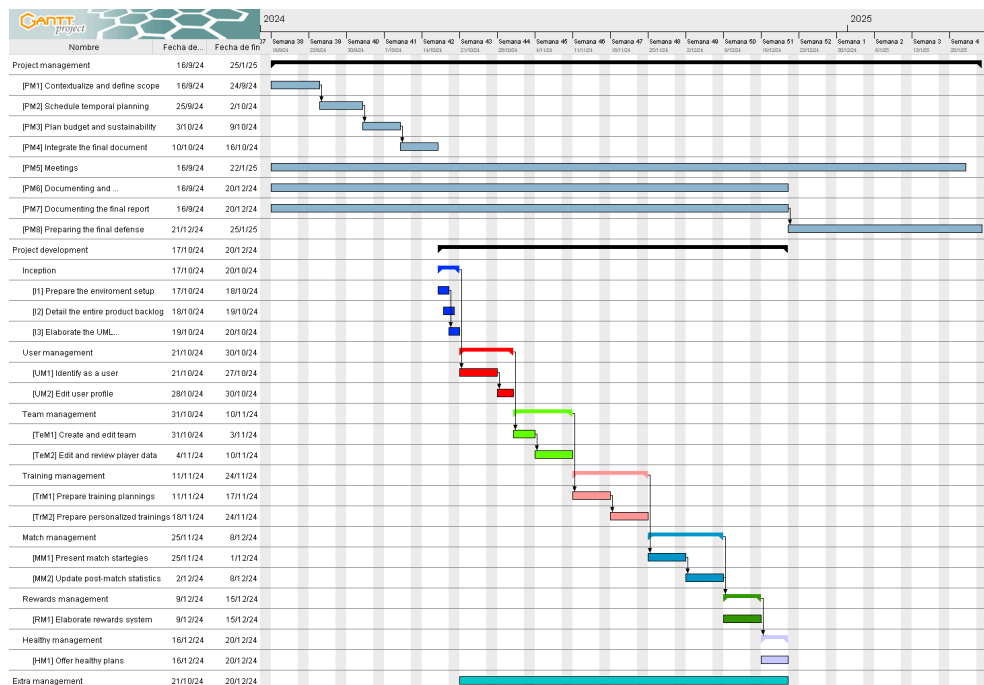


Figure 34. Gantt chart. Source: The Author.

6.3 Initial budget management

In order to correctly analyze the costs of project development, it is advisable to identify and calculate the following factors: personnel costs, generic costs and possible anomalies of the costs. Apart from this, a possible mechanism to ensure a better management control will be presented.

6.3.1 Staff costs by activity

To be able to identify the costs by activity, first of all we need to determine the different roles⁵ that will participate in the project development. That is why in the following Table 6, the roles with their cost per hour are presented and the social security costs⁶ are included as well.

All this information has been gathered from Glassdoor [20] and contrasted with Talent [21].

⁵ I have divided the Software developer into frontend and backend and considered that the Project manager is who does the practical tasks of the Thesis writer before he writes them down on the report.

⁶ It is calculated with the sum of 30% as the contribution rate.

Role	Cots (€/h) Gross salary	Cost (€/h) Social security
Project manager (HRPM)	22,40	29,12
Software architect (HRSA)	27,60	35,88
Software frontend developer (HRSFD)	18,23	23,70
Software backend developer (HRSBD)	20,83	27,08
Tester (HRT)	14,06	18,28

Table 6. Roles with gross salaries plus social security per hour. Source: The Author.

Based on the hourly salary, the following Table 7 shows the cost of each task according to the role that performs it. After these calculations, the sum of all of them is the final personnel costs per activity (CPA).

Code	Task	Workload	Hours					Cost (€)
			HRPM	HRSA	HRSFD	HRSBD	HRT	
PM1	Contextualize and define scope	20 h	20					582,40
PM2	Schedule temporal planning	15 h	15					436,80
PM3	Plan budget and sustainability	15 h	15					436,80
PM4	Integrate the final document	20 h	20					582,40
PM5	Meetings	20 h	15	5				616,20
PM6	Documenting and noting during the process	-						-
PM7	Documenting the final report	70 h	70					2.038,40
PM8	Preparing the final defense	30 h	30					873,70
I1	Prepare the environment setup	8 h		1	2	3	2	201,10
I2	Detail the entire product backlog	7 h	5	2				217,40
I3	Elaborate the UML diagram and its	10 h		7		3		332,4

	DDBB							
UM1	Identify as a user	25 h			8	13	4	614,80
UM2	Edit user profile	15 h			5	7	3	362,90
TeM1	Create and edit team	30 h			12	13	5	727,84
TeM2	Edit and review player data	35 h			15	15	5	853,10
TrM1	Prepare training plannings	30 h			15	12	3	735,30
TrM2	Prepare personalized trainings	40 h			13	22	5	995,26
MM1	Present match strategies	40 h			13	22	5	995,26
MM2	Update post-match statistics	30 h			10	17	3	752,20
RM1	Elaborate rewards system	25 h			10	13	2	625,60
HM1	Offer healthy plans	30 h			14	13	3	738,70
	Extra management	25 h						To Be Determined
TOTAL		540 h	190	15	117	153	40	13.718,56 + extras

Table 7. Personnel costs per activity (CPA). Source: The Author.

6.3.2 Generally calculated costs

General costs are those that are not directly related to any task and human resources, although they are necessary for the development of the project.

Hardware

First of all, there are the hardware components that I will use. To go on with the amortization that I want to apply in the following 4 years and make an approximation of the money that is going to cost us this hardware, the following calculation has been presented:

(Hardware € cost / (4 years * 251 working days * 6 hours a day)) * (93 days dedicated to the project * 6 hours a day)

Hardware	Cost (€)	Amortització (€)
PCComponentes assembled PC	1.263,13	117,00
Monitor (LG ULTRAGEAR)	109,00	10,10
Keyboard (Nemskill)	34,63	3,21
Mouse (G203 LIGHTSYNC)	19,00	1,76
TOTAL		132,07

Table 8. Hardware components costs. Source: The Author.

Software

In second place, here are the software components that I will use. As it can be seen in the following table, because all the software resources that I will use are free to use, the total cost is 0.00 €.

Software	Cost (€)
GanttProject	0,00
Zotero	0,00
Visual Studio Code	0,00
Github	0,00
Taiga	0,00
Atenea & Racó	0,00
Gmail	0,00
Google Meet	0,00
Google Chrome	0,00
TOTAL	0,00

Table 9. Software components costs. Source: The Author.

Additional expenses

The **Internet** costs 19,90 €/month with Movistar (0,0267 per hour) [22]. If I work 540 project hours and in 520 of them I will make use of it, Internet will cost me $520 * 0,0267 = 13,89$ €.

My PC consumes 169 W (0,169kW) [23], I will work with it for 520 hours and **electricity** costs 0,1851 €/ kWh [24]. Total kW per hour: $0,169\text{kW} * 520 \text{ h} = 87,88\text{kWh}$. It will cost me $87,88\text{kWh} * 0,1851 \text{ €/kWh} = 16,27$ €.

I have estimated that the **room** from where I work will cost me initially 412,80 €/month, as I have calculated with the mean of the first 20 rooms found in Idealista [25]. However, I will round it up to 450,00 € due to house location. So $450,00 \text{ €} * 5 \text{ months of work} = 2.250,00$ €.

Taking all this information into account, in Table 10 it can be seen a brief summary of the generic costs (CG).

IT Infrastructure	Cost (€)
Hardware	132,07
Software	0,00
Internet	13,89
Electricity	16,27
Workspace	2.250,00
TOTAL	2.412,23

Table 10. Generic cost (CG). Source: The Author.

6.3.3 Contingencies

It is important to calculate some % in order to cover my back due to possible things that might happen. In IT, the values usually tend to vary between 10% and 20%, so I will use 15%:

Concepts	Cost (€)
Personnel costs per activity (CPA)	13.716,56
Generic costs (CG)	2.412,23
Total	16.128,79
Contingency	2.419,32

Table 11. Contingency cost. Source: The Author.

6.3.4 Incidentals

In the project, there might be some unexpected events related to the risks from previous sections. That is why in Table 12, there is calculated the money that will cost the plans B of the obstacles according to the probability of happening, the role that would solve them and the time that they will invest in doing so:

Risk	Time (h) and role	Base cost (€)	%	Final cost (€)
Bugs	10 HRSFD, 15 HRSBD	$237 + 406,2 = 643,20$	75	482,40
Delivery deadlines	15 HRPM	436,8	50	218,40
Inexperience with technologies	5 HRSFD, 5 HRSBD	$118,5 + 135,4 = 253,90$	25	63,48

Technical problems	1 HRPM, 1 HRSA, 1 HRSFD, 1 HRSBD, 1 HRT	29,12 + 35,88 + 23,7 + 27,08 + 18,28 = 134,06	25	33,52
TOTAL				797,80

Table 12. Incidentals cost. Source: The Author.

6.3.5 Total cost estimation

Finally, a brief summary of the total cost is presented (being this rounded up):

Concept	Cost (€)
Personnel costs per activity (CPA)	13.716,56
Generic costs (CG)	2.412,23
Contingencies	2.419,32
Incidentals	797,80
TOTAL	19.346,00

Table 13. Total final cost. Source: The Author.

6.4 Management control

In this section I present the supervision of the possible deviations that the project may experience regarding the estimated costs. To do so, I will use some management control indicators.

At the end of the project, these indicators, their deviation and their positive or negative values will help in deciding if I will need to use the contingencies calculations.

$$[\text{Cost deviation per task} = \sum_{i=1}^n (\text{Expected cost hours} - \text{Real cost hours}) * \text{Cost per hour}]$$

being n the roles that perform the task.

The sum of the *Cost deviation per task* of all the project tasks will show us their total deviation:

$$[\text{Total tasks cost deviation} = \sum_{i=1}^n \text{Cost deviation per task}] \text{ being } n \text{ the tasks of the project.}$$

$$[\text{Total hours deviation} = \text{Expected hours} - \text{Real hours}]$$

$$[\text{Total incidentals deviation} = \text{Expected incidental cost} - \text{Real incidental cost}]$$

$$[\text{Total cost deviation} = \text{Total cost estimation} - \text{Real cost}]$$

6.5 Sustainability report

6.5.1 Self-assessment

The survey carried out has helped me to put into context the knowledge I had about the three dimensions to be studied: environmental, social and economic and served as a solid foundation for, not only learning new things about them but also being aware of how to approach these aspects.

First of all, addressing the **environmental dimension**, in the CI subject I was able to see, understand and put into practice methods to prevent obsolete hardware from generating harmful and irreparable waste, so it is a concept that I have already seen and been familiarized with by working with it.

Referring to the **social dimension**, in the PES subject I was able to develop an application that recommended events to people taking into account their interests and personal preferences, in addition to making a social chat to enhance communication between people. The main objective of it was to improve user experience from a social point of view, which we achieved satisfactorily.

Finally, although I have done subjects where I needed to elaborate economic plans for applications and do cost estimations (as in GEP), the **economic dimension** is the one in which I will have to work the most and in which I have the most room for improvement, as I consider it to be the weakest one.

With all these being said, I not only consider that the survey done is useful for the project development but for the jobs where I can work and the environment where I will move, their characteristics and labor implications referring to the environmental, social and economic dimension as well. Therefore, I hope to enrich myself much more in these dimensions throughout the development of the project, just as I have been doing during my university life with those that I have encountered.

6.5.2 Economic dimension

Regarding PPP: Reflection on the cost you have estimated for the completion of the project. I have estimated the human, material, general and other important costs in order to obtain the general cost of the project. These costs have been obtained using precise calculations about the information collected.

Regarding Useful Life: How are currently solved economic issues (costs...) related to the problem that you want to address (state of the art)?

How will your solution improve economic issues (costs ...) with respect other existing solutions?

As I mentioned in previous sections, nowadays coaches do not have a useful way of solving their daily football problems in an efficient way. There are currently solutions on the market, but they are not as economic and useful as they could be.

On the other hand, the solution I propose expects to be free to use and without any external paying program. However, as I consider that it will be an excellent option, I expect people to be interested in it and maybe include advertisements in the future or sell it so that what it costs to make it is profitable.

6.5.3 Environmental dimension

Regarding PPP: Have you estimated the environmental impact of the project?

The whole environmental impact will be given by the electricity consumed, which is the only weak point in the development of the application in this area.

Regarding PPP: Did you plan to minimize its impact, for example, by reusing resources?

One of the few things I can consider in this project is to program and work in the most efficient way possible so that it does not involve a large expense of energy and resources.

Regarding Useful Life: How is currently solved the problem that you want to address (state of the art)? How will your solution improve the environment with respect other existing solutions?

Presently, and like I have mentioned just before, coaches tend to write on papers, so if they wanted to solve this problem they would have to use less paper (an aspect that would be a problem for their tactics) or use a computer.

That is why, with my solution, they would continue using the computer but in a more profitable way and I hope to minimize the constant usage and throwaway of paper.

6.5.4 Social dimension

Regarding PPP: What do you think you will achieve -in terms of personal growth- from doing this project?

First of all, it represents the biggest project I have been involved in, so it will be a useful way of understanding new technologies but, more importantly, ways and methods of working in projects. Secondly, it would mean a lot to if the project is realistic and used by real coaches.

Regarding Useful Life: How is currently solved the problem that you want to address (state of the art)?

How will your solution improve the quality of life (social dimension) with respect other existing solutions?

The current solutions are not as user friendly as they could be. They tend to be difficult to use and impractical.

My application is intended to be used by all types of coaches of all ages and with different technology skills. It offers a more realistic and functional solution.

This is why I expect it to represent an improvement for the lives of the users who use it.

Regarding Useful Life: Is there a real need for the project?

Finally, I truly believe that there is a real need for the development of the project.

This idea arose from the need that I had when I was training with little children and it was impractical for me to have the team information in the mobile phone notes, for example.

That is why I believe that the idea of the project not only arises for the sake of it but as a necessity for a specific group of the society.

7. Deviations from the initial plan

7.1 Task deviations

Due to my decision to prioritize the completion and perfection of the most important and relevant tasks of the project, which were related to the basis of the management of teams, players, training sessions, matches,..., I have decided to dispense with the implementation of the following tasks:

RM1: Elaborate rewards system and **HM1: Offer healthy plans.**

Apart from the fact that I had already considered them as extra tasks from the beginning as they are not so related to the main content of the application and since the people involved in carrying them out were the same for all the tasks, no extra time nor workers was needed. So the no completion of these tasks has not been a big problem in terms of temporal and economic deviations and has not been significantly noticed.

Furthermore, this opens the doors to the proposal of carrying out a possible future work that starts from the implementation of these extra tasks and continues towards a possible application that focuses on player empowerment, without taking so much into account organization and team management.

7.2 Temporal deviations

Since the project's temporal planning was done till today's date, there have been a few temporal deviations that have made me invest a little more time than I had planned.

Those tasks that caused some deviations were:

TeM1: Create and edit team. Since I had to add all the usable clubs from which the teams are part of. (**4 hours** the 6th of November)

TrM1: Prepare training plannings. Since I had to research, choose and add manually all the exercises needed to create the trainings. (**6 hours** the 7th of November)

Then, I had to spend almost **20 hours** on **bugs** and **2,5 hours** on **inexperience with technologies** from those that were already planned.

Using the management controls indicators for the tasks, we can obtaining the following results:

Expected hours - Real hours = Total hours deviation

325 - 335 = -10 hours

As I have stated in my personal sprint plannings and sprint reports, there have been some adjustments in the tasks that should have been done in each Sprint, so in the following section I present Sprint's updates:⁷

Sprint 1 (21/10 - 3/11) : UM1, UM2

Sprint 2 (4/11 - 17/11) : TeM1 (+- TrM1)

Sprint 3 (18/11 - 1/12) : TeM2

Sprint 4 (2/12 - 15/12) : TrM1, TrM2

Sprint 5 (16/12 - 29/12) : MM1

Sprint 6 (30/12 - 12/12) : MM2

However, since I had plenty of time to cover my back, these updates and the extra 10 hours do not affect anything other than the salary of the workers who will have to invest in them.

Moreover, the workers have distributed the total work hours in the new sprint. So they have not worked so intensely in a shorter time but rather in a calmer manner over a longer period of time.

7.3 Economic deviations

The previous temporary delays are reflected in more investment of money.

The *TeM1: Create and edit team* deviation took 2 hours from the Software backend developer (27.08 €/hour), and 2 hours from the Software frontend developer (23,70 €/hour), resulting in a total of 101,56 € of extra costs for this task.

The *TrM1: Prepare training plannings* deviation took 5 hours from the Software backend developer (27.08 €/hour), and 1 hour from the Software frontend developer (23,70 €/hour), resulting in a total of 159,10 € of extra costs for this task.

Total of **260,66 €**.

Using the management controls indicators, we can confirm these previous results:

$$[\text{Cost deviation per task} = \sum_{i=1}^n (\text{Expected cost hours} - \text{Real cost hours}) * \text{Cost per hour}]$$

being n the roles that perform the task.⁸

Cost deviation TeM1: $((12 - 14) * 23,70) + ((13 - 15) * 27,08) + ((5 - 5) * 18,28) = -101,56 €$

Cost deviation TrM1: $((15 - 16) * 23,70) + ((12 - 17) * 27,08) + ((3 - 3) * 18,28) = -159,10 €$

⁷ To update this information I have used Table 5.

⁸ To do these calculations, I have used Table 6 and Table 7.

The sum of the *Cost deviation per task* of all the project tasks will show us their total deviation, as shown before:

[**Total tasks cost deviation** = $\sum_{i=1}^n \text{Cost deviation per task}$] being n the tasks of the project.

$$(101,56) + (-159,10) = -260,66 \text{ €}$$

[**Total incidentals deviation (in bugs)** = **Expected incidental cost - Real incidental cost**]⁹

$$482,40 - (10 \text{ hours} * 23,70 + 10 \text{ hours} * 27,08) = -25,4 \text{ € in bug incidentals.}$$

[**Total incidentals deviation (in inexperience with technologies)** = **Expected incidental cost - Real incidental cost**]

$$63,48 - (2,5 \text{ hours} * 23,70) = 4,23 \text{ € still left for inexperience with technology incidentals.}$$

$$\text{Total incidental deviation} = -25,4 + 4,23 = -21,17 \text{ € in incidentals deviation.}$$

[**Total cost deviation** = **Total cost estimation - Real cost**]

$$19.346,00 - (19.346,00 + 260,66 + 21,17) = +281,83 \text{ € of total cost deviation for the project.}$$

7.4 Final assessment

On the one hand, in terms of project development and planning, I am quite proud, as I personally believe that I have been able to implement the main tasks on time and in a satisfactory manner. I made effective use of the extra time I had intentionally planned for this purpose and adhered closely to the plan.

Nevertheless, I still feel a lingering regret for not having been able to implement the healthy plans and rewards. Although looking at it from a positive point of view, it serves as a starting point and inspiration for future projects.

On the other hand, I consider that the economic planning has been refined and closely traced to the final results after the completion of the project. Without being an expert on this topic, I personally believe that I have managed myself well enough, as evidenced by the managing controllers for the economic deviations. That said, there is always room for improvement.

⁹ To do these calculations, I have used Table 12

8. Laws and regulations

The project must be done taking into account the following legislations:

Ley Orgánica de Protección de Datos Personales y Garantía De Los Derechos Digitales:

This law ensures the protection of personal information and guarantees digital rights for citizens.

The project uses the data from the user for a specific, legitimate and relevant purpose, as it is used exclusively for the app necessities.

General Data Protection Regulation:

This law ensures the protection of personal data of individuals within the European Union.

It applies to the processing of personal data by automated means and to manual processing of personal data that forms part of a filing system.

As stated in the previous law, the project uses the data from the user for a specific, legitimate and relevant purpose, as it is used exclusively for the app necessities.

In order to do the user registrations and logins, the system needs to write and read from the database, as it needs to save and recover the username and password from the user.

Since this application wants the coach-player relationship to be as similar as possible as in real life, player characteristics such as height, weight, ID, email, telephone, nationality, photo, average heart rate and drug use level will be saved in order to understand the needs of each player in a more specific way and even prevent negative situations from happening.

To avoid problems with these laws, the following principles will be assured:

1. **Informed consent:** We will obtain consent from the parents of the player or the player itself (+18), explaining which, how and why the data will be used.
2. **Necessary data:** We will save and work with the essential and necessary data for the application.
3. **Transparency right:** The parents of the player or the player itself (+18) will be informed about how their data is handled and processed.
4. **Communication right:** The parents of the player or the player itself (+18) will be informed about important and relevant changes regarding their data.
5. **Data security and use:** As the majority of data will already have been provided from the parents to the club (name, surnames, birthdate,...) it will be stored in databases strictly from the application and there will be unique authentication for each coach, being able to add only the data available from the club, previously provided by the parents when registering the player to be able to play for that club or information provided by the parents/players themselves.

9. Knowledge integration

9.1 Knowledge acquired from the courses

With the development of both the project and the app, I have been forced to work with and on things that I have already worked on in the Computer Engineering degree but this time I had to do it on my own. That is why I have had to work in an organized way and apply all the knowledge acquired from the 4 years that I have spent in FIB.

Apart from the personal methodology that I have used and how I have faced the problems, there are some areas and subjects of the Computer Engineering degree that have been used directly or indirectly.

1. **PRO1, PRO2, EDA:** These subjects have helped me to know languages, techniques and things related to programming. From having a base to being a bit more advanced.
2. **BD, DBD:** These subjects have helped me understand the concept of a DB, each component, how they work and how to interact with it.
3. **IES, AS:** These subjects helped me to design the most accurate architecture models, conceptual schemas and diagrams to represent the project fields involved in the application as well as to write a reusable and flexible program with the most precise pattern techniques.
4. **ER, GPS:** These subjects have helped me to do the vast majority initial parts of the project: from specification phases, identification of stakeholder, objectives, requirements, use cases and context study to useful software project management techniques and their temporal plannings.
5. **PROP, PES:** These subjects both apply object-oriented programming techniques, but what they helped me the most is in how to work in a big programming project in teams and how to distribute, organize, update and review the code.
6. **ASW:** This subject has helped me to design a web interface to represent the proposed project solution and interact with it using API communication.

9.2 Software Engineering specialization justification

As this project is part of the Software specialization, there are plenty of concepts related with it that justify this choice:

1. **Requirements and specification phases:** In this project, different phases of a software project have been carried out. From requirements study and choice to use cases, user stories, and other specification concepts; I have gone through these phases at the beginning of the project in order to have a good base from which to start before designing the application.
2. **Conceptual and design modeling and databases:** In order to use and treat all the data involved in the project, a correct modeling of the idea has been a must. That is why, conceptual model diagrams with its restrictions, constraints, components and relations have been done to represent these ideas. Apart from this, a database has been used to apply the ideas from the design model and interact with them, allowing to store and retrieve important data that the application has to use and that is represented in the modelization of the project.
3. **Web page and API:** The project solution has been presented in a well structured and organized web page format, allowing users to interact with it through API requests and access to its services and functionalities.
4. **Software design:** To make the application scalable, flexible and reusable, a quality software design has been applied. The methodologies used, its patterns and how they are applied have been an important aspect of this project.
5. **Testing:** In order to ensure the correctness of the application code, different tests and in different situations have been carried out to guarantee the correct performance of all the project functionalities.

9.3 Technical competences

When the project had to be proposed with its title, supervisor and specialization, the technical competencies that would be addressed during the project had to be specified. Below, these are presented, specifying their level and explaining how they will be carried out.

1. **CES1.1:** Develop, maintain and evaluate complex and/or critical software systems and services. [**In depth**]
Like most software systems, this project requires a design composed of different parts that interact with each other, such as the specification and study of components and the subsequent design. To organize and carry out this, it is essential to have notable knowledge in development, maintenance and evaluation in these complex systems.

2. **CES1.2:** Solve integration problems based on strategies, standards and available technologies. [**A little**]

In a smaller scope, this project proposes solutions to integration problems such as the use of common protocols in strategies and updated documentation.

3. **CES1.3:** Identify, evaluate and manage the potential risks associated with the construction of software that could be presented. [**Enough**]

As done in GEP and throughout the project, all the possible risks that may occur have been identified, evaluated and managed when necessary, presenting possible solutions and plan Bs.

4. **CES1.6:** Manage Databases (CIS4.3) [**In depth**]

As the conceptual models need to be represented in some kind of way, a database has been used in order to do so. Its creation, initialization and interaction has been a must to represent all the data and manage its information.

5. **CES1.7:** Control quality and design tests in software production. [**Enough**]

To verify the integrity of the code and test its functionalities and correct operation, tests have been carried out. With these tests, it was possible to avoid code updates with errors and find other fails in functionalities that had holes in their behavior.

This is how a correct development of the code was assured.

6. **CES1.9:** Demonstrate understanding in the management and governance of software systems. [**Enough**]

Similar to **CES1.1**, this competence involves having proper governance throughout the project lifecycle. It is basically a summary of how an appropriate software system should evolve through all the phases of the project and maintain its standards and goals.

7. **CES2.1:** Define and manage the requirements of a software system. [**In depth**]

As stated in previous sections, before starting the application all the requirements from the system had to be presented, specified and taken into consideration. As in most software systems, before starting the application development some rules must be presented on which to start and on which the development of the project can be based on, so it follows a common path.

8. **CES2.2:** Design appropriate solutions in one or more application domains, using software engineering methods that integrate ethical, social, legal and economic aspects. [**A little**]

This project wants to be done in an ethical way and respecting all social and legal boundaries, that is why some aspects such as disrespect or offenses have no place in this application and are treated inside of it, such as the prohibition of the use of offensive usernames.

10. Conclusions

10.1 Objectives conclusions

Before starting the project, many objectives were stated in order to know what it wanted to be achieved when developing it. Once the development of the application has been completed, it can be seen how all the objectives have been satisfactorily achieved:

1. **Centralized data:** The application offers access to data about teams, players, training, statistics and other information related to teams created by coaches using the platform.
2. **Performance analysis:** The application offers automatic updates for information related to teams, which helps in presenting strategies for matches. It also allows manual updates related to players, which helps to understand their performance for match call-ups and lineups and to identify their needs for training sessions (whether personalized or not).
3. **Insights and conclusions generation:** The application offers insights using statistical information when approaching matches in order to understand teams' situations, enhancing the strengths and addressing weaknesses.
4. **Training and match management:** The application offers the possibility to create and update training to suit coaches preferences, as well as matches to simulate outcomes and potential scenarios based on real league situations.

Finally, in addition to these more specific objectives, the main one has been achieved: to develop a fully functional web application used as a helpful tool for football coaches in areas such as training, matches, team information, and statistics, facilitating the effective management of teams.

10.2 Future project

My project started from the basic problem of the poor organization that coaches had nowadays. From notes on mobile phones to hand calculations of players' minutes. That is why I decided to develop an application that would solve this problem.

As I have progressed in the project, I have seen aspects that I would have liked to improve or carry out in a different way. These revolve around the coach-player relationship. That is why a future project could be an application not only for coaches but for all the people who are part of the same team. In this way, and simulating what a *WhatsApp* group could be, for example, the communication, relationship and even performance of the entire team would be improved.

The main points that the application could have are:

1. **Team chat:** With it, all the players (or their parents) and the coach could have fluid communication and be available at all times.
2. **Rewards system:** The application could have a reward system to reward the most prepared players who put in the most effort.
3. **Healthy management:** Apart from the basic aspects, the software could have a section that recommended healthy plans depending on player necessities.
4. **Administrative management:** To improve the administrative management of the team and the club, there could be an area that would store and allow all data related to this to be processed.

10.3 Final conclusions

After the completion of this project, which not only encompasses the development of the application, but also the initial idea, the GEP month, the progress of the specification, the meetings with the director and much more, I can reach the following conclusions:

First, I personally believe that the result of the application has been very satisfactory and of a high level. The key points have been met, the objectives have been successfully completed, good use of the present technologies has been made and errors have been properly treated. Moreover, the project has followed a logical and planned evolution, while specific points of the career and the Software branch have been addressed.

In addition, I also wanted to comment on the sustainability of the project at a technical, economic and material level.

As the application has been developed using scalable and modern technologies such as NodeJS, React and Firebase, the system can grow, be correctly maintained and evolve.

Apart from the correct economic plan presented, accessibility to the application is free, with the possibility of adding advertisements to obtain economic benefits. However, this addition would not affect in any notable way the app usage.

Since the application uses the technologies mentioned before and it is efficiently coded, it is optimized, reducing the computational resources required to run it and the energy consumption.

And finally, speaking in a more personal way, I am proud of myself and the work carried out.

What at first seems like a mountain of challenges and long-term objectives, has become a result of which I am very satisfied and that I have enjoyed carrying out.

I have worked with technologies that I already knew and others that were new to me. I have used software methodologies that were familiar to me and others that I had to learn from. Everything from patience and the desire to develop an application that was the result of a well-done project.

Thanks to the perseverance and help of my director, Xavier Oriol Hilari, we have been able to not only complete the final degree project, but also enjoy the process.

11. References

- [1] Admin, G. R. F. (2018, March 7). 10 common problems every coach will encounter and the Solution. *Team Grassroots*.
<https://www.teamgrassroots.co.uk/10-common-problems-every-coach-will-encounter-solution/>
- [2] BOE-A-2018-16673 Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. (n.d.). Retrieved 23 September 2024, from
<https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- [3] Non-functional requirement. (2024). In *Wikipedia*.
- [4] Bakić, L. (2024, April 12). *Workload Management: Ultimate Guide to Efficiency*. Productive.
<https://productive.io/blog/workload-management/>
https://en.wikipedia.org/w/index.php?title=Non-functional_requirement&oldid=1245895117
- [5] Atlassian. (2021, September 7). *6 popular project management methodologies and what they're best suited for*. Work Life by Atlassian.
<https://www.atlassian.com/blog/project-management/project-management-methodologies>
- [6] *Agile Project Management Explained*. (n.d.). Office Timeline. Retrieved 24 September 2024, from
<https://www.officetimeline.com/project-management/agile>
- [7] ausum-editor. (2024, January 10). Scrum: Qué es y por qué es una de las metodologías ágiles. *Ausum Cloud: Servicios y Soluciones Cloud*.
<https://ausum.cloud/scrum-metodologia-agil-mas-popular-en-empresas/>
- [8] *What is Kanban?* | Agile Alliance. (2017, April 4). <https://www.agilealliance.org/glossary/kanban/>
- [9] Bernsen, R. (2023, March 10). *OpenProject: La alternativa de código abierto a Taiga*. OpenProject.org. <https://www.openproject.org/es/blog/openproject-alternativa-a-taiga/>
- [10] *How to Use Git Branches & Buddy to Organize Project Code—SitePoint*. (2019, July 9).
<https://www.sitepoint.com/use-git-branches-buddy/>
- [11] s.r.o, B. S. (n.d.). *GanttProject: Free project management tool for Windows, macOS and Linux*. GanttProject. Retrieved 2 October 2024, from <https://www.ganttproject.biz>
- [12] Zotero | *Your personal research assistant*. (n.d.). Retrieved 2 October 2024, from
<https://www.zotero.org/>
- [13] *Visual Studio Code—Code Editing. Redefined*. (n.d.). Retrieved 2 October 2024, from
<https://code.visualstudio.com/>
- [14] *Build software better, together*. (n.d.). GitHub. Retrieved 2 October 2024, from
<https://github.com>
- [15] *Taiga: Your opensource agile project management software*. (n.d.). Retrieved 2 October 2024, from
<https://taiga.io/>

- [16] *El Racó* | *Facultad de Informática de Barcelona*. (n.d.). Retrieved 2 October 2024, from <https://www.fib.upc.edu/es/la-fib/servicios-tic/el-raco>
- [17] *Gmail: Correo electrónico sin coste, privado y seguro* | *Google Workspace*. (n.d.). Retrieved 2 October 2024, from <https://www.google.com/intl/es/gmail/about/>
- [18] *Google Meet*. (n.d.). Retrieved 2 October 2024, from <https://meet.google.com/landing>
- [19] *Google Chrome: El navegador web rápido y seguro diseñado para ti*. (n.d.). Retrieved 2 October 2024, from https://www.google.com/intl/es_es/chrome/by-environment/chrome-browser/
- [20] *Sueldos de la empresa*. (n.d.). Glassdoor. Retrieved 8 October 2024, from <https://www.glassdoor.es/Sueldos/index.htm>
- [21] *Salario en España—Salario Medio*. (n.d.). Talent.com. Retrieved 8 October 2024, from <https://es.talent.com/salary>
- [22] *Descubre nuestras ofertas: Fibra Óptica 300Mb*. (n.d.). Retrieved 9 October 2024, from https://amp.movistarfusion.es/landings/?custom=19546392618_kwd-297505476138&sem=900500186&tc_alt=5408&n_okw=movistar%20internet_b_c_154595249165_19546392618_kwd-297505476138_&gad_source=1&gclid=Cj0KCQjw05i4BhDiARIsAB_2wfB9NPdWxhT-Npu2HXCxxtv8HG7V2eEVpFydWC1yVaQAIT5UPaiEU2AaApXOEALw_wcB&gclsrc=aw.ds
- [23] *Choose Your Parts—PCPartPicker*. (n.d.). Retrieved 8 October 2024, from <https://pcpartpicker.com/list/>
- [24] *Precio de la tarifa de luz por horas HOY | 08 Octubre 2024* . (n.d.). Retrieved 8 October 2024, from <https://tarifaluzhora.es/>
- [25] *Habitacions de lloguer a Terrassa, Barcelona—Idealista*. (n.d.). Retrieved 8 October 2024, from <https://www.idealista.com/ca/alquiler-habitacion/terrassa-barcelona/>